



UNIVERSITÉ
BISHOP'S
UNIVERSITY

Computer Science Department
CS 596 - Research Topics in Computer Science
Winter 2024

Final project: Exploring Recurrent Neural Networks (RNNs) with PyTorch

Students:

Group : FP B

Cicéron Ovide Aissoun -- Ahmad Issa

Dr. Mohammed Ayoub Alaoui Mhamdi

Bishop's University

Sherbrooke, Qc, Canada

malaoui@ubishops.ca

Exploring Recurrent Neural Networks (RNNs) with PyTorch

- We implemented LSTM , RNN model using PyTorch.
- The code builds an LSTM (Long Short-Term Memory) model to predict stock prices.
- We download and preprocess stock yahoo finance data, as well as prepare the data for the LSTM model.
- We use Yahoo! Finance's stock prices dataset
- Yfinance is open-source tool that uses Yahoo's publicly available APIs and is intended for research and educational purposes.

Method

- The code demonstrates how to build and train an LSTM model for stock price prediction using PyTorch.
- We used SGD optimizer and multiple hidden layers
- We also shows how to tune the model's hyperparameters to improve its performance.
- We define a set of hyperparameters to be tuned:
 - Learning rates: 0.001, 0.01, 0.1
 - Hidden layer sizes: 50, 75, 100
 - Batch sizes: 10, 16, 32, 64

Method (ctd)

- **LSTM Model Definition**

- The StockLSTM class is defined, which inherits from nn.Module (PyTorch's base class for neural network models).
- The class has the following attributes:
 - input_size: the size of the input features
 - hidden_layer_size: the size of the hidden layer
 - output_size: the size of the output (stock price prediction)
 - lstm: the LSTM layer
 - linear: a linear layer for the output
 - hidden_cell: the initial hidden and cell states for the LSTM

Method (ctd)

- **Data Downloading and Processing**

- The `download_and_process_stock_data` function is defined to download stock data from Yahoo Finance for a given ticker, start date, and end date.
- The function returns the stock data with only the 'Close' column.

Method (ctd)

- **Data Preparation**

- The `prepare_data` function is defined to preprocess the stock data for the LSTM model.
- It scales the data using the `MinMaxScaler`, and then creates the input sequences (X) and output labels (Y) for the model.
- The input sequences have a fixed lookback window, which determines how many previous days of data are used to predict the next day's stock price.

Method (ctd)

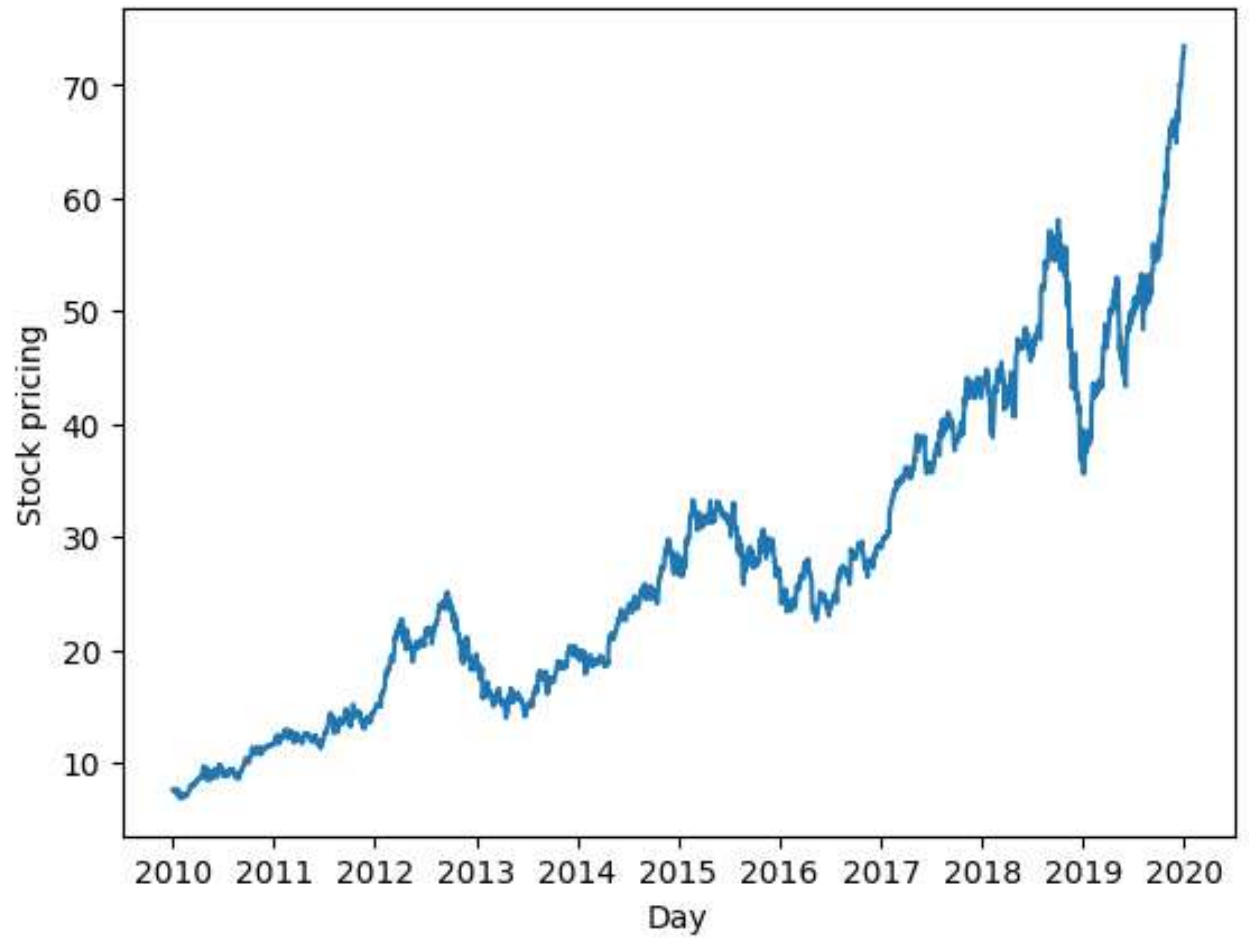
- **Hyperparameter Tuning** (as described above)
- **Training and Evaluation**
 - We train the LSTM model for each combination of the hyperparameters.
 - During training, the model's loss is calculated using the Mean Squared Error (MSE) loss function, and the model is optimized using Stochastic Gradient Descent (SGD).
 - The training loss is plotted for each hyperparameter configuration.

Method (ctd)

- **Model Evaluation**

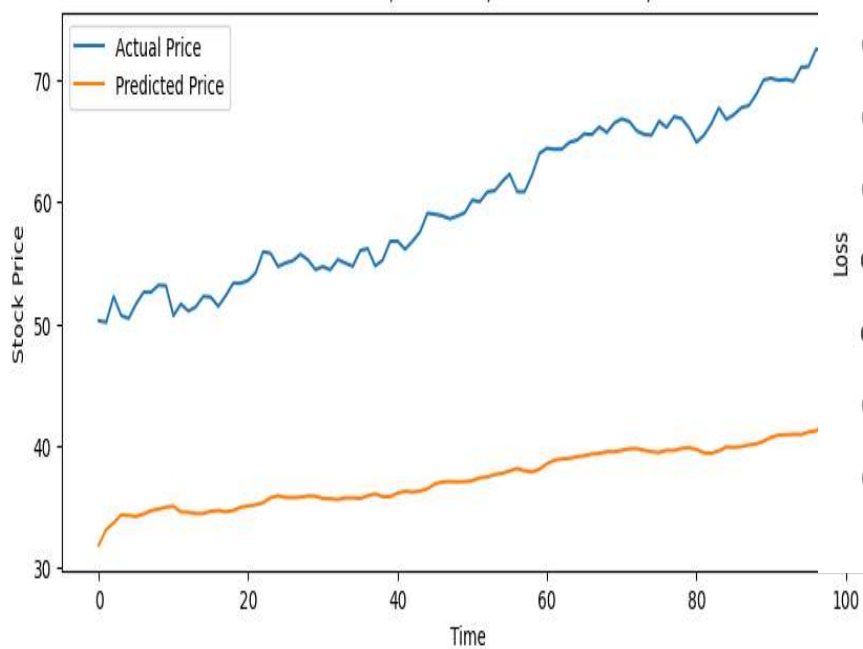
- After training, we evaluate the model's performance on the test set.
- It makes predictions using the trained model and compares the predicted prices to the actual prices.
- The actual and predicted prices are plotted for each hyperparameter configuration.

Results

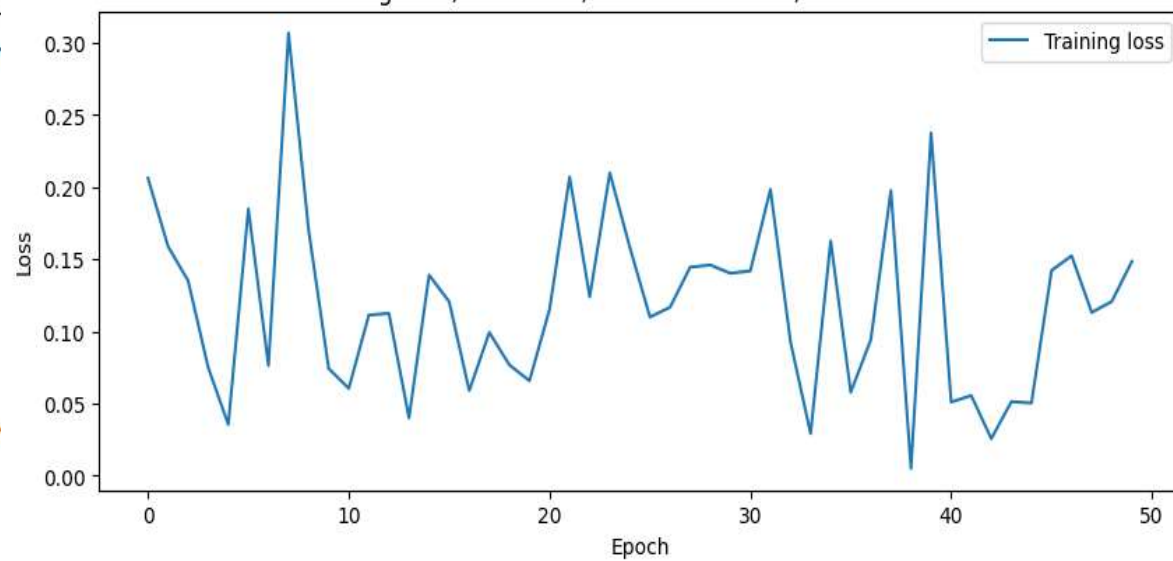


Results

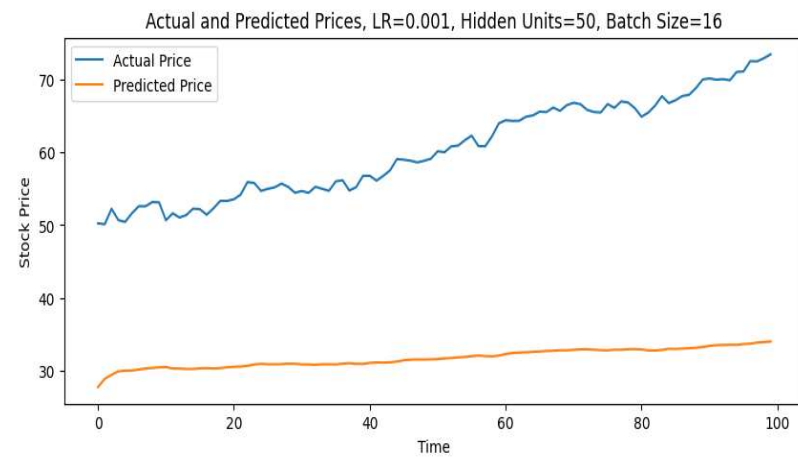
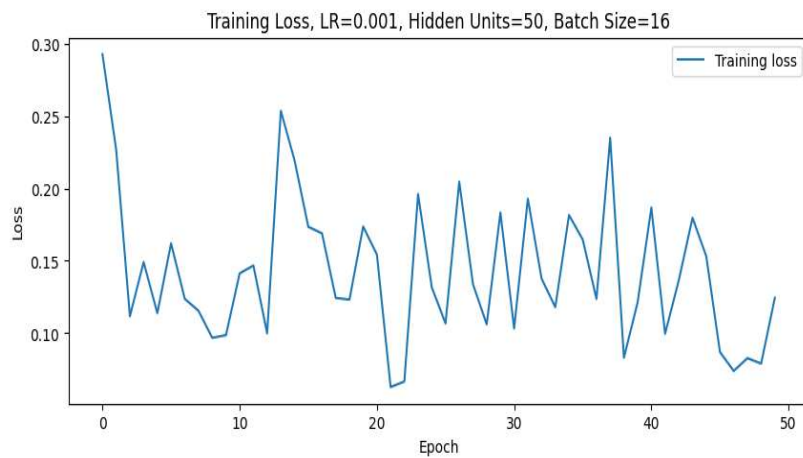
Actual and Predicted Prices, LR=0.001, Hidden Units=50, Batch Size=10



Training Loss, LR=0.001, Hidden Units=50, Batch Size=10

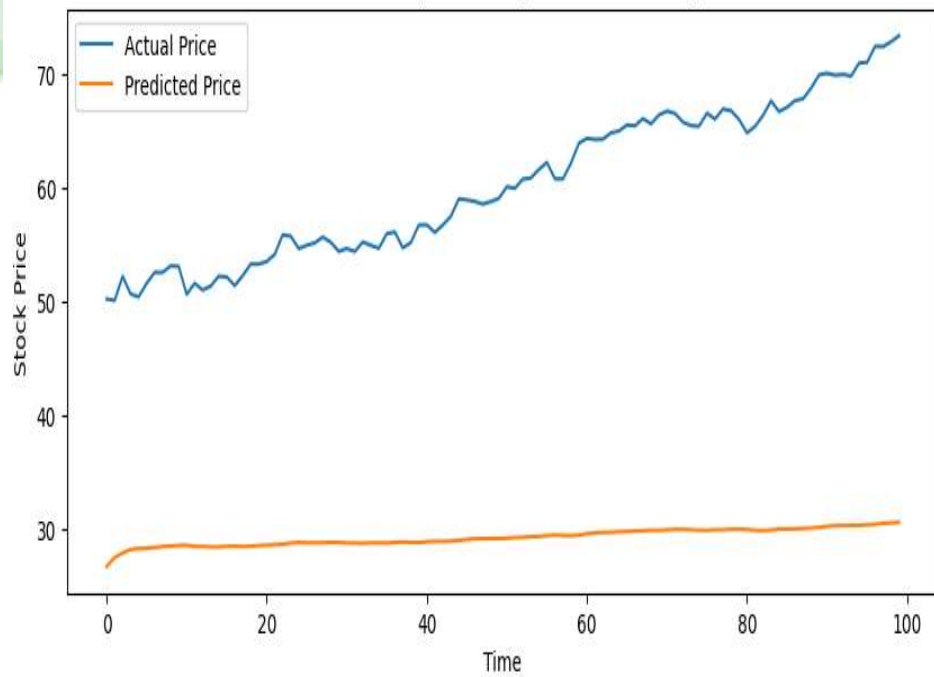


Results

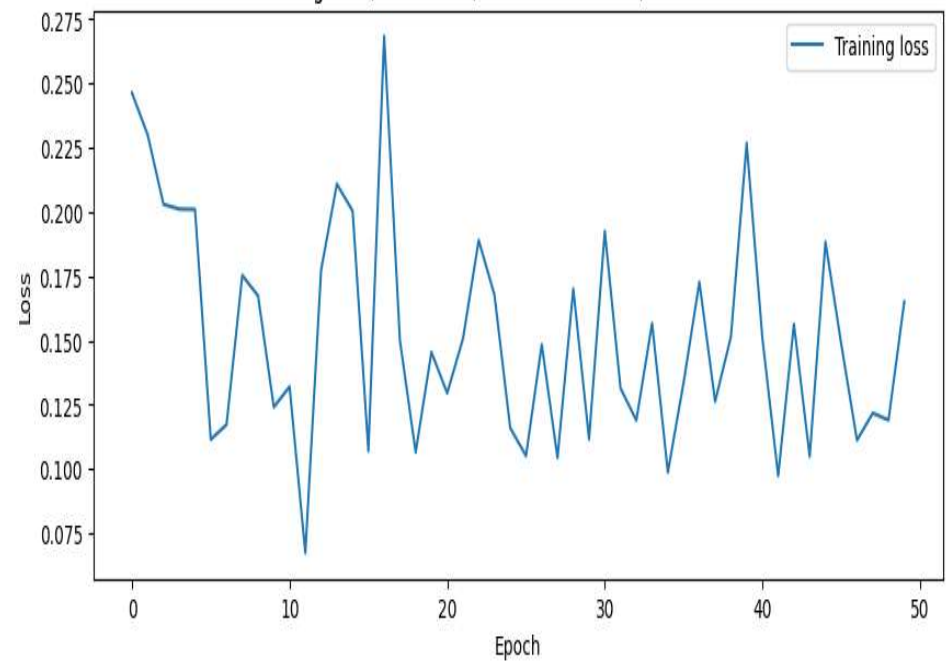


Results

Actual and Predicted Prices, LR=0.001, Hidden Units=50, Batch Size=32

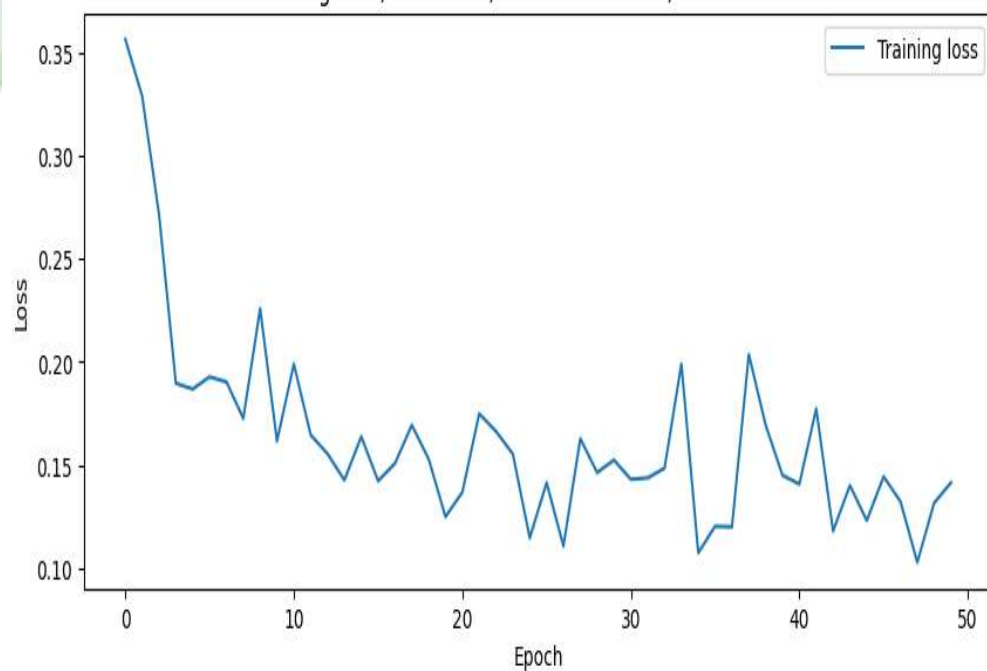


Training Loss, LR=0.001, Hidden Units=50, Batch Size=32

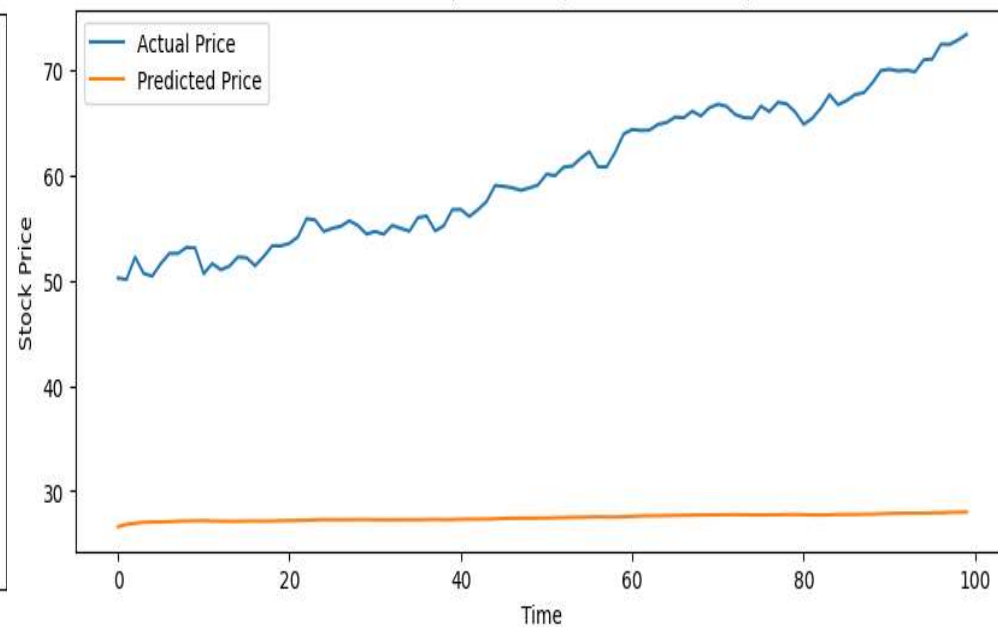


Results

Training Loss, LR=0.001, Hidden Units=50, Batch Size=64

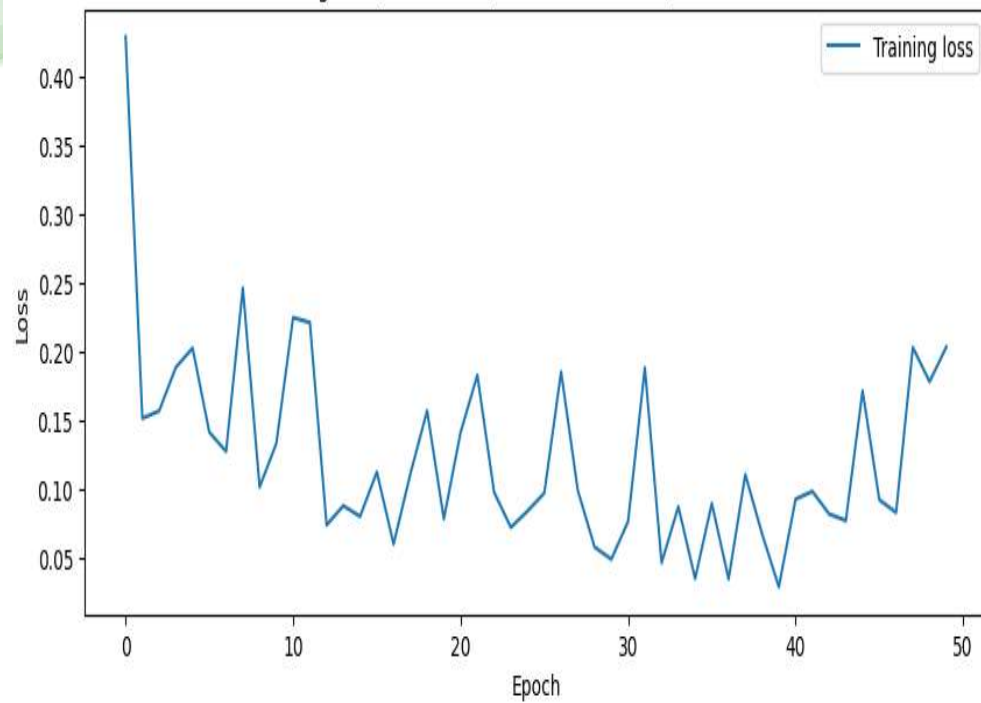


Actual and Predicted Prices, LR=0.001, Hidden Units=50, Batch Size=64

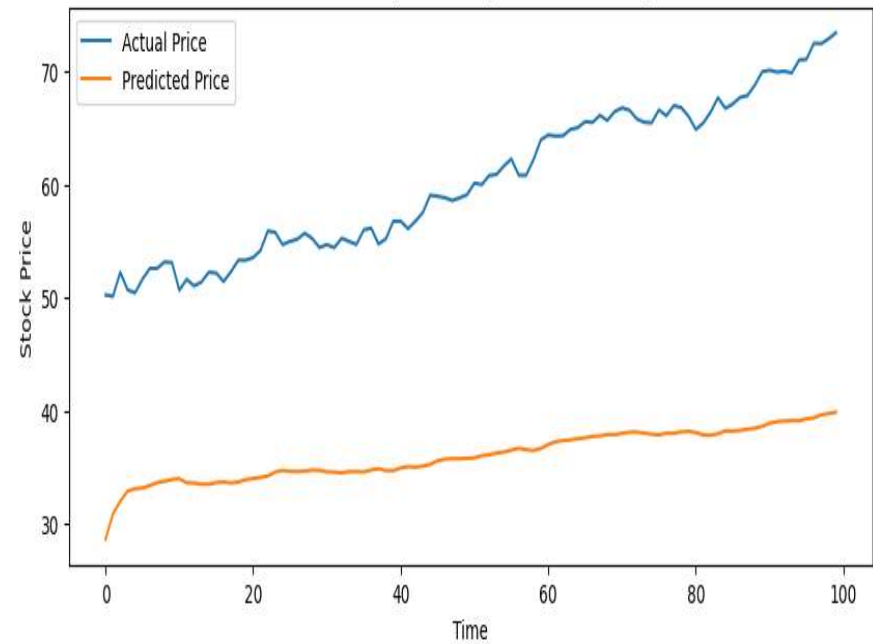


Results

Training Loss, LR=0.001, Hidden Units=75, Batch Size=10

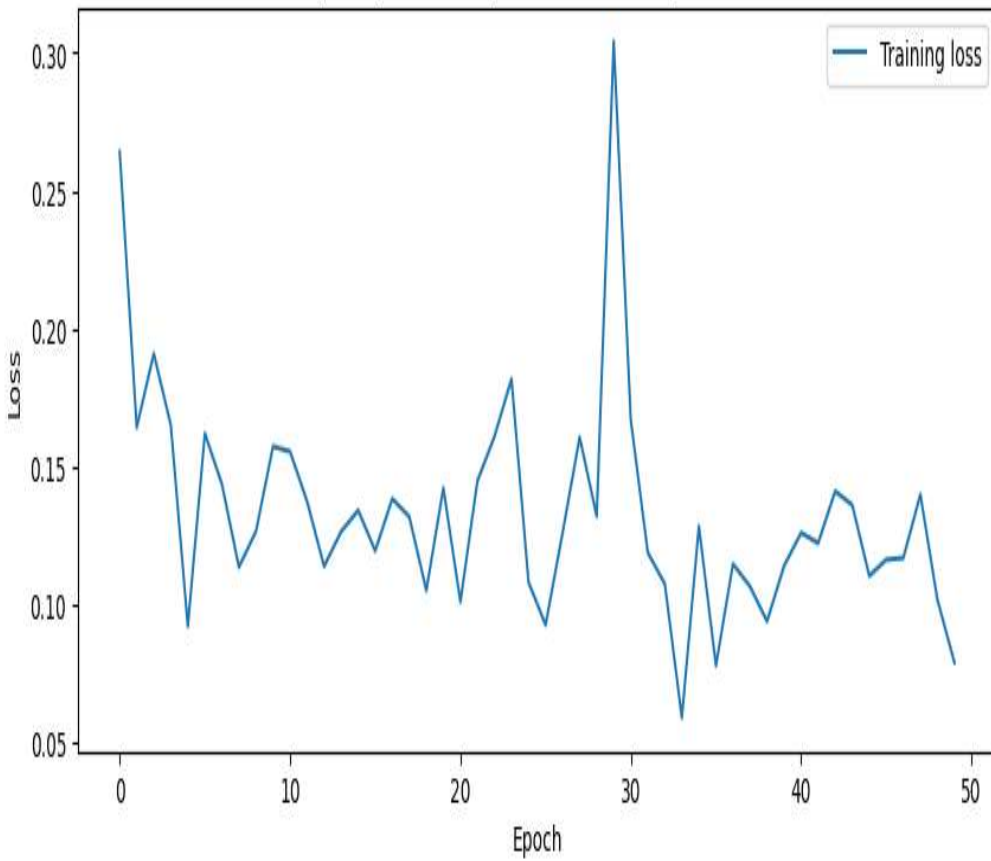


Actual and Predicted Prices, LR=0.001, Hidden Units=75, Batch Size=10

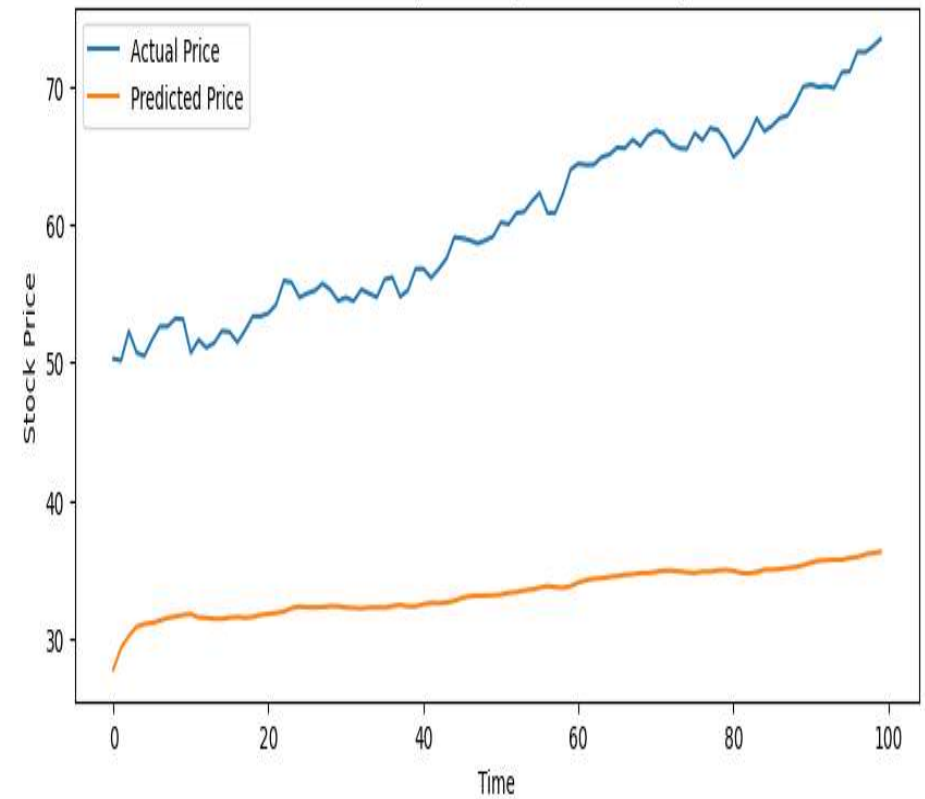


Results

Training Loss, LR=0.001, Hidden Units=75, Batch Size=16

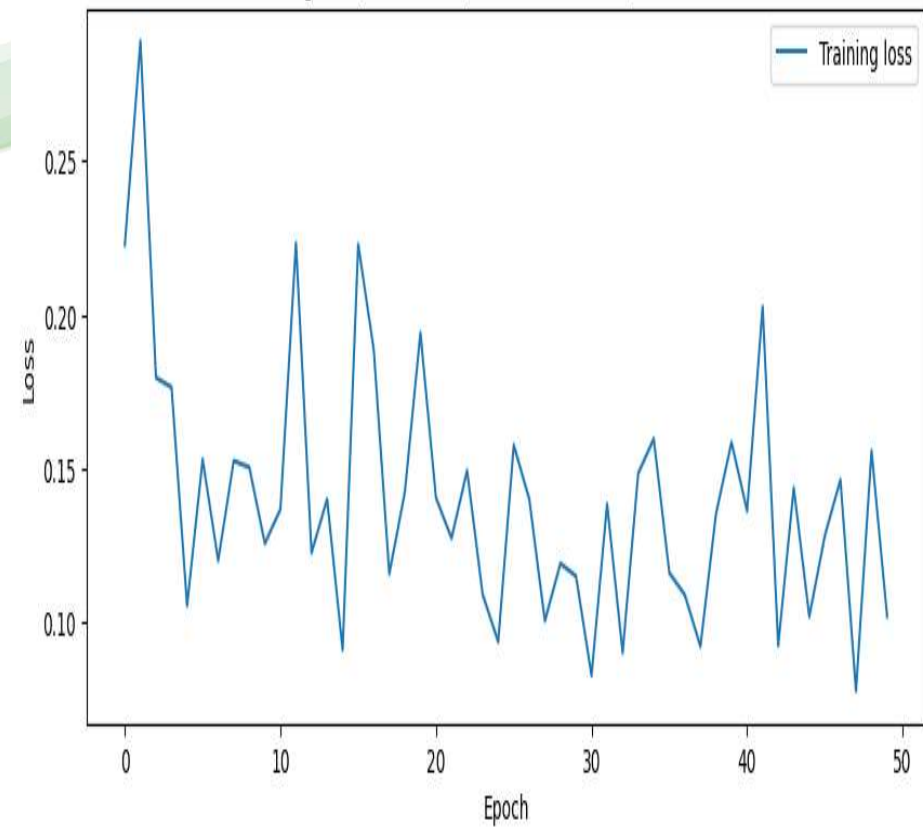


Actual and Predicted Prices, LR=0.001, Hidden Units=75, Batch Size=16

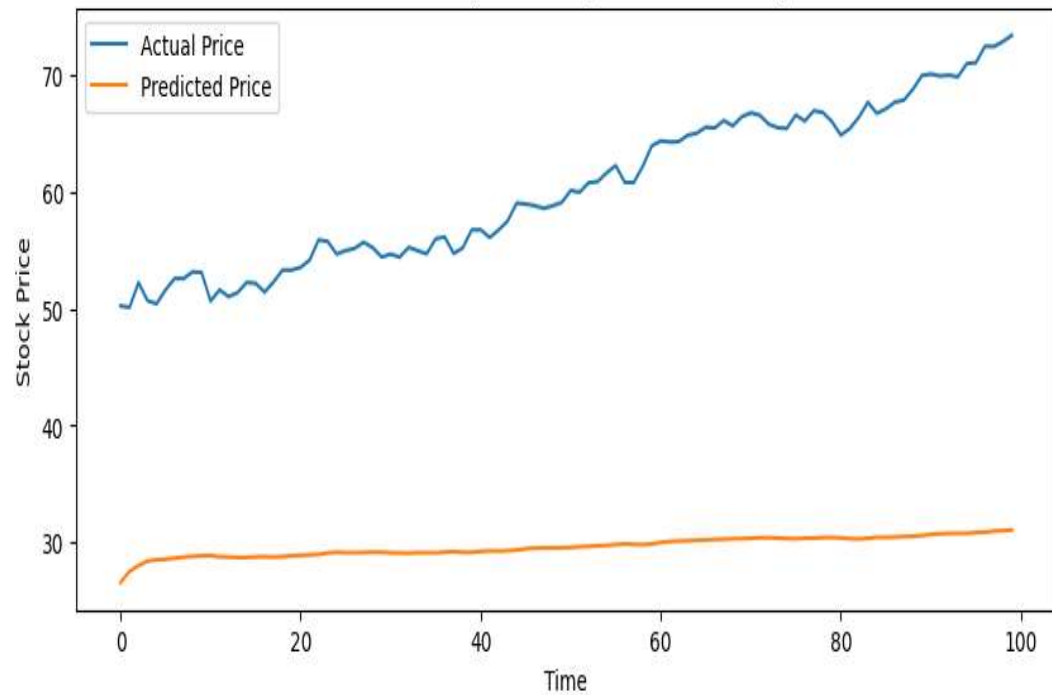


Results

Training Loss, LR=0.001, Hidden Units=75, Batch Size=32

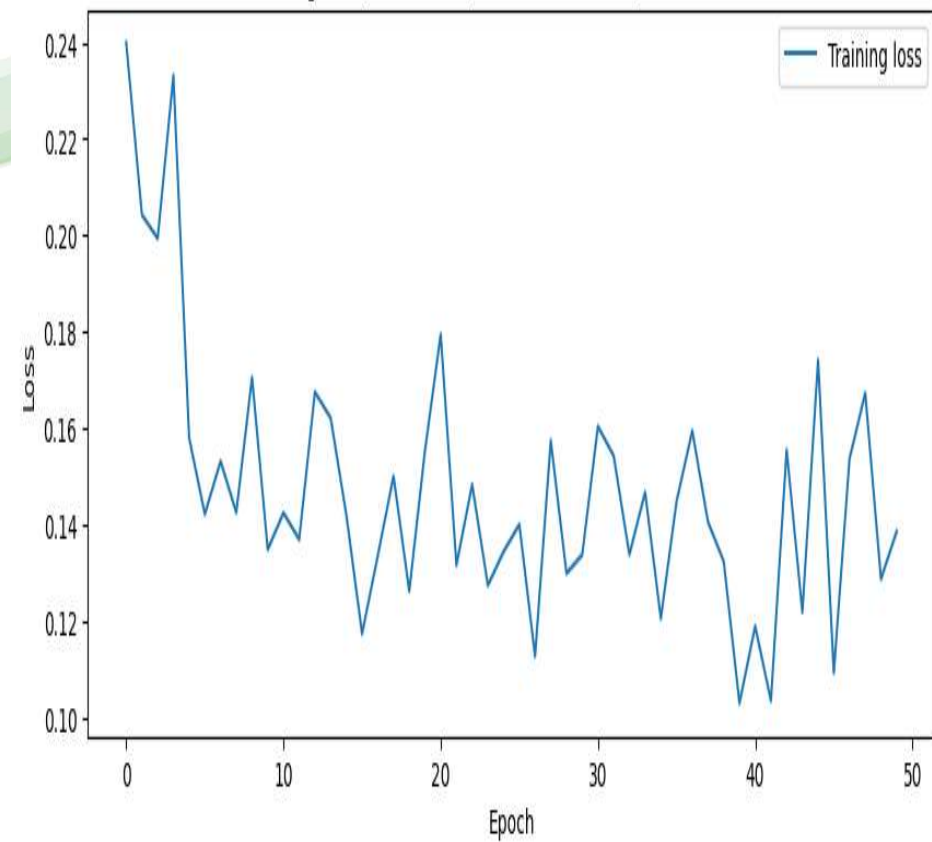


Actual and Predicted Prices, LR=0.001, Hidden Units=75, Batch Size=32

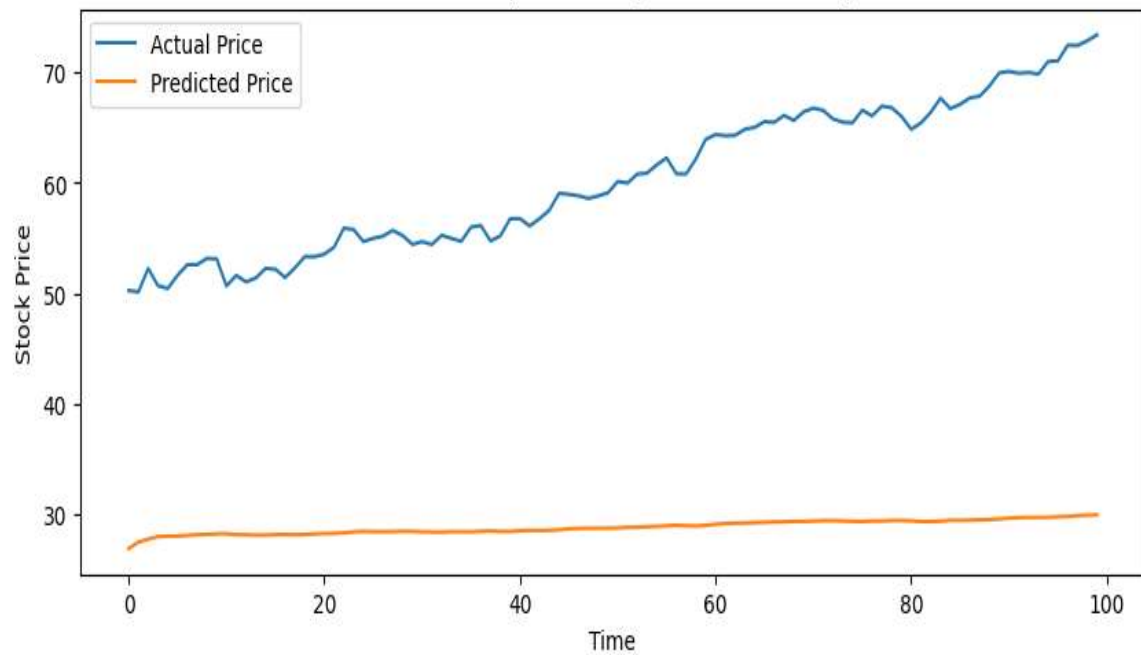


Results

Training Loss, LR=0.001, Hidden Units=75, Batch Size=64

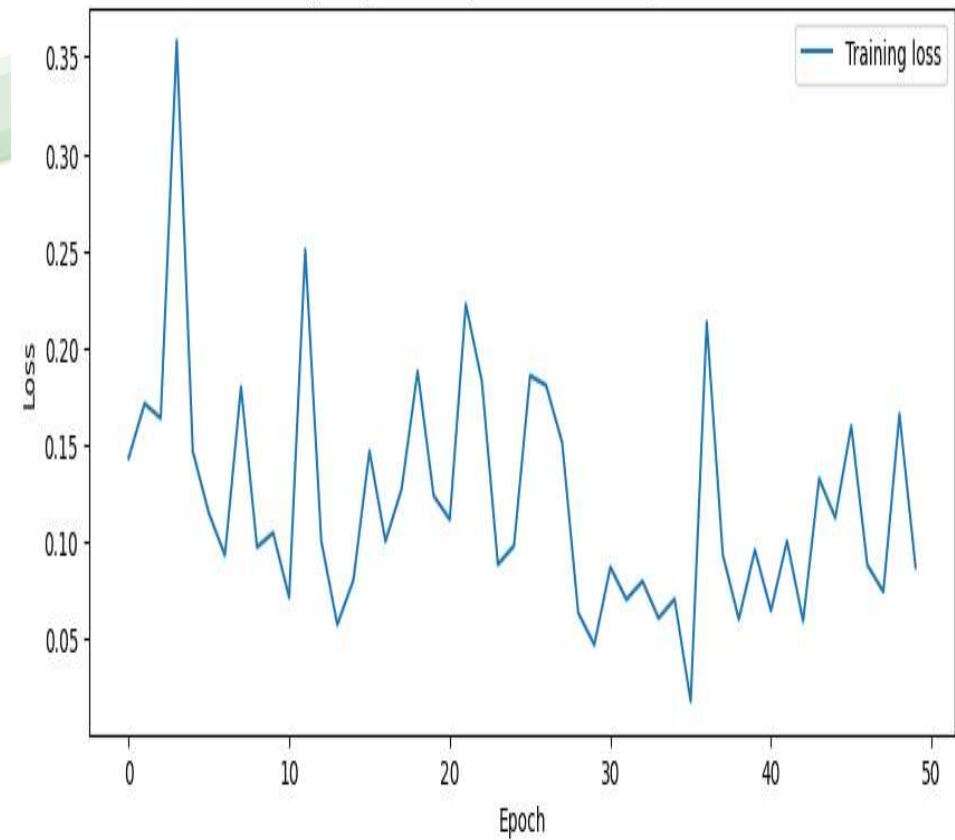


Actual and Predicted Prices, LR=0.001, Hidden Units=75, Batch Size=64

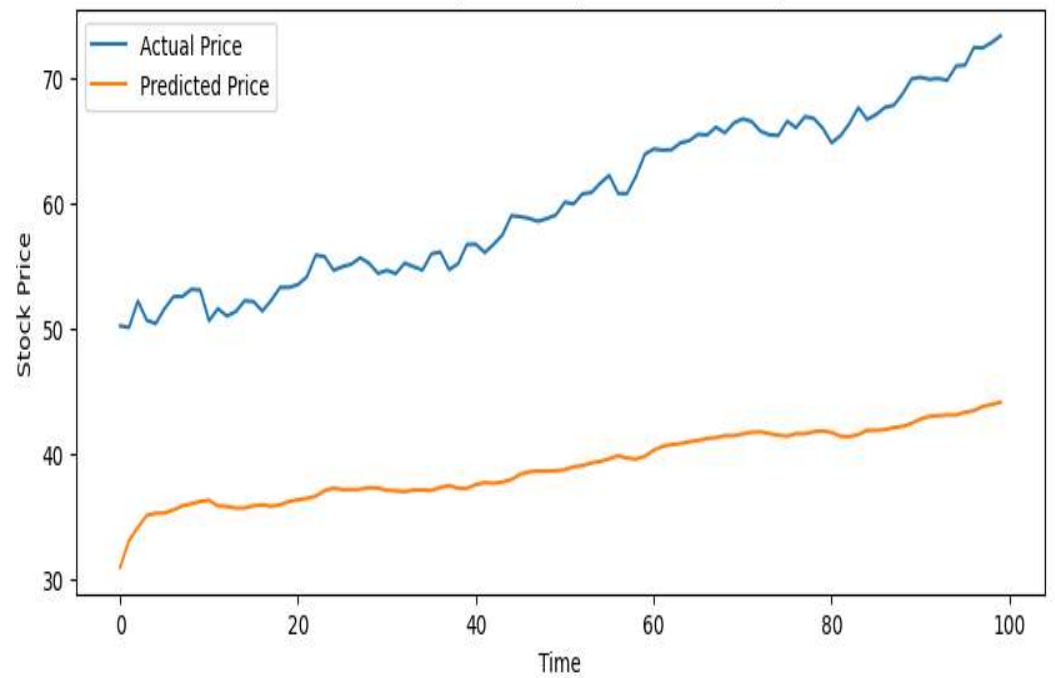


Results

Training Loss, LR=0.001, Hidden Units=100, Batch Size=10

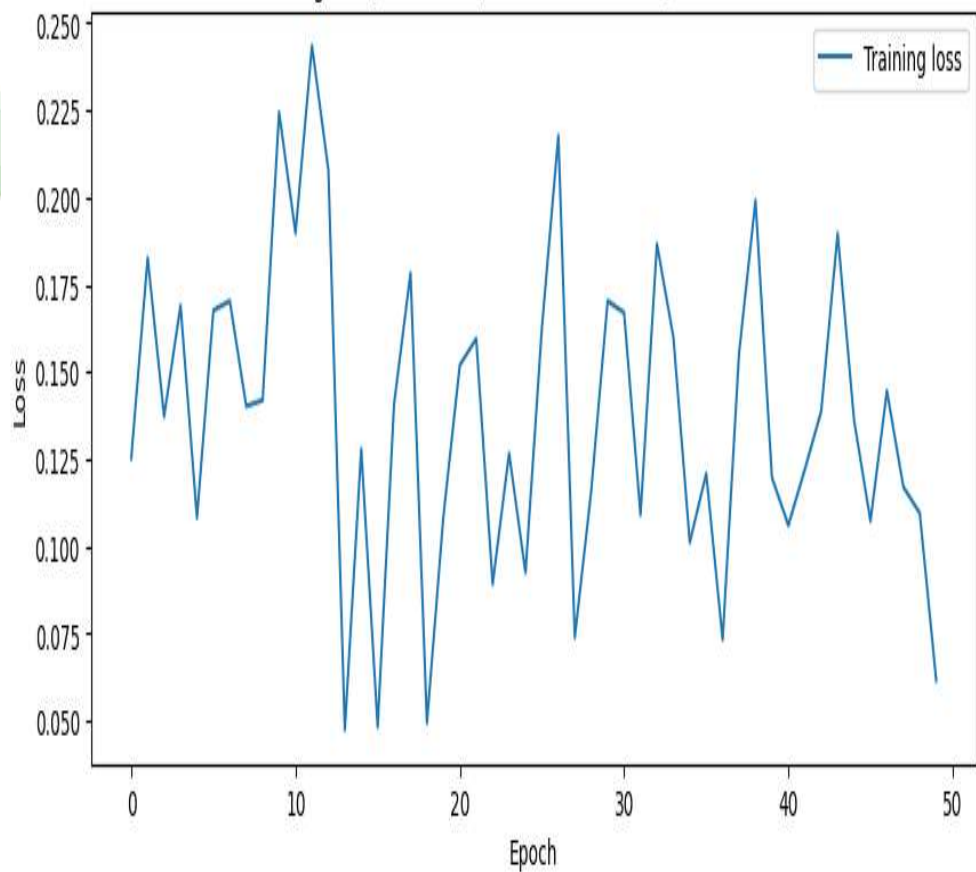


Actual and Predicted Prices, LR=0.001, Hidden Units=100, Batch Size=10

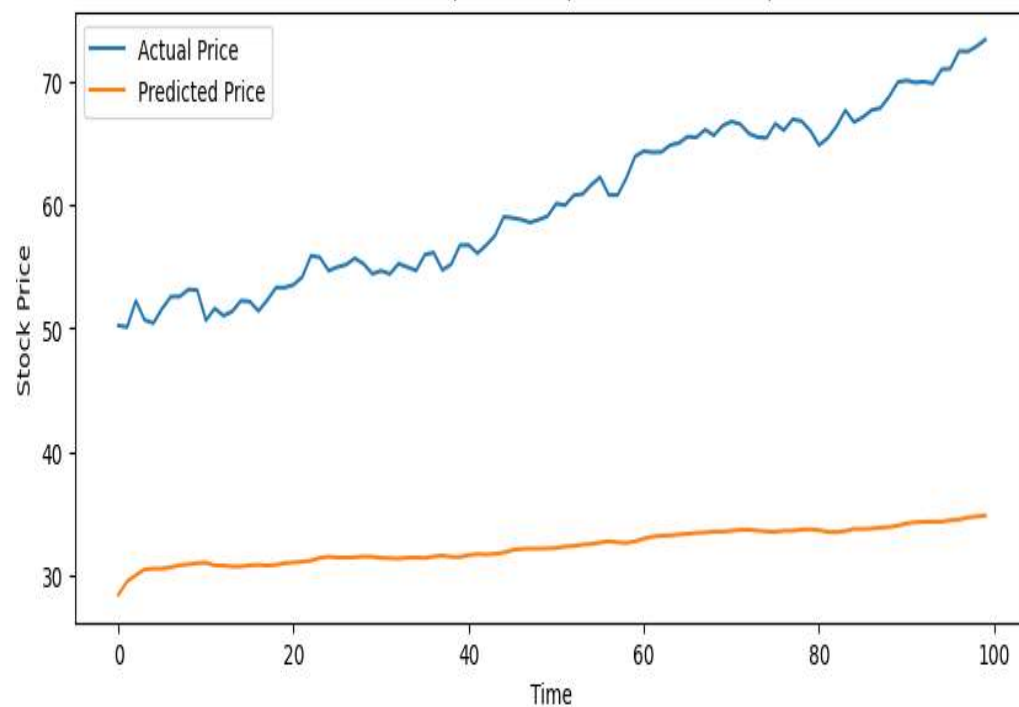


Results

Training Loss, LR=0.001, Hidden Units=100, Batch Size=16

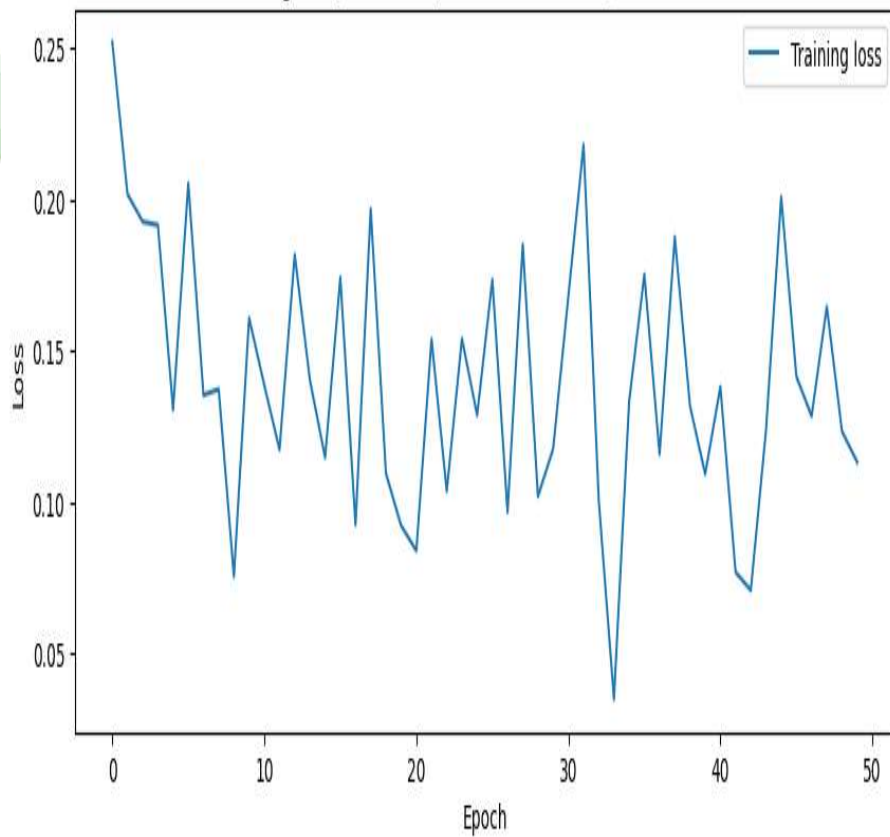


Actual and Predicted Prices, LR=0.001, Hidden Units=100, Batch Size=16

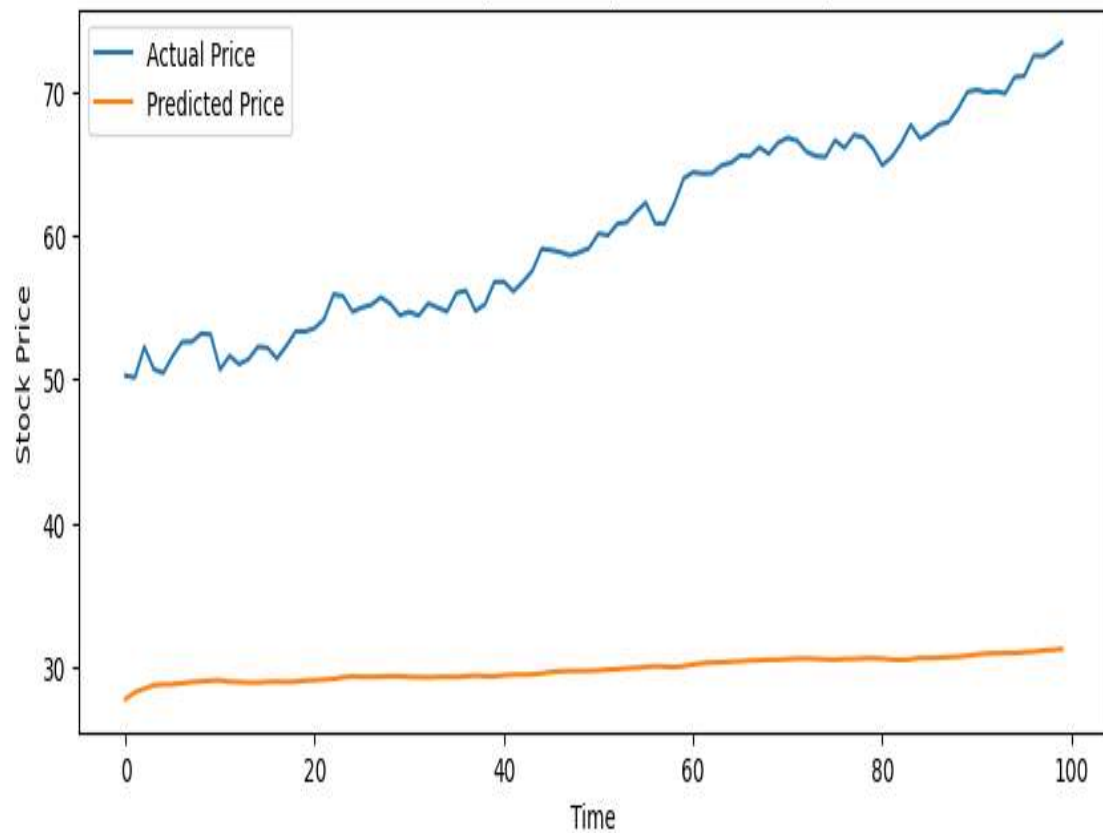


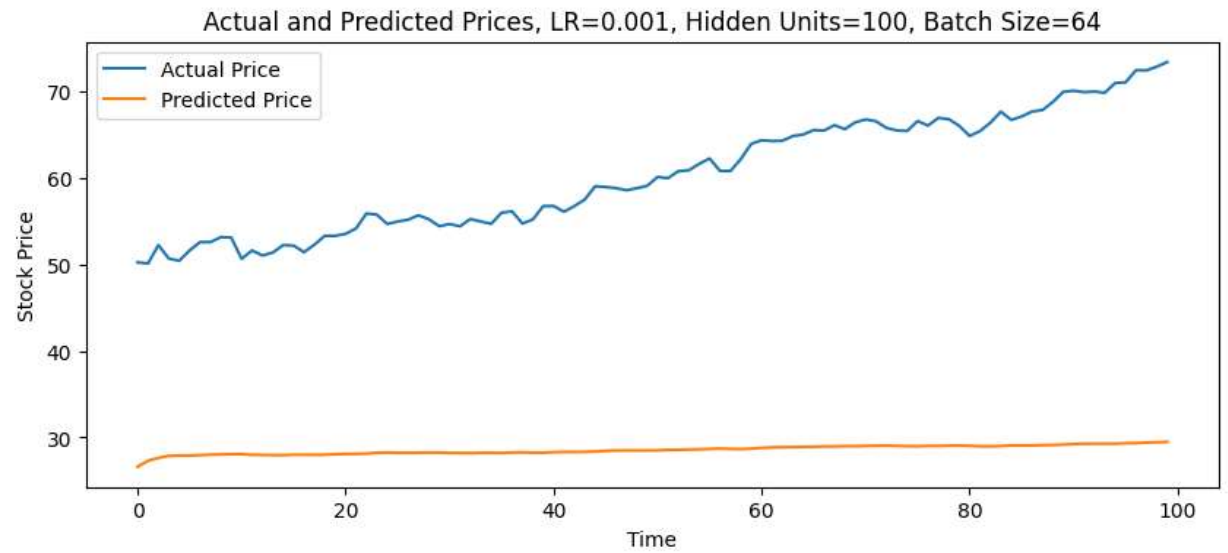
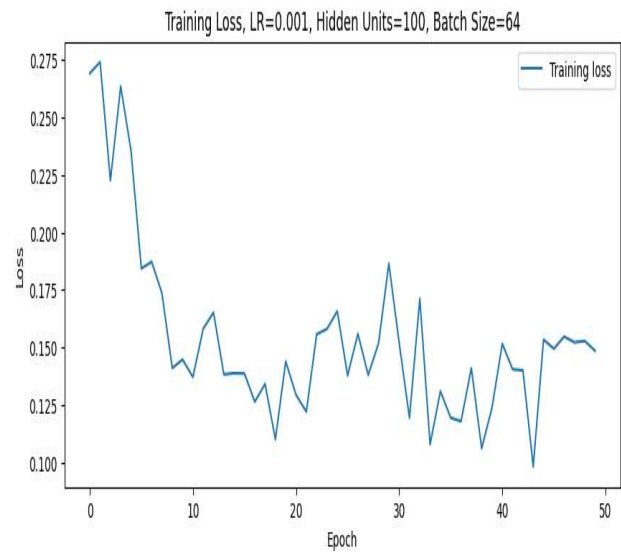
Results

Training Loss, LR=0.001, Hidden Units=100, Batch Size=32

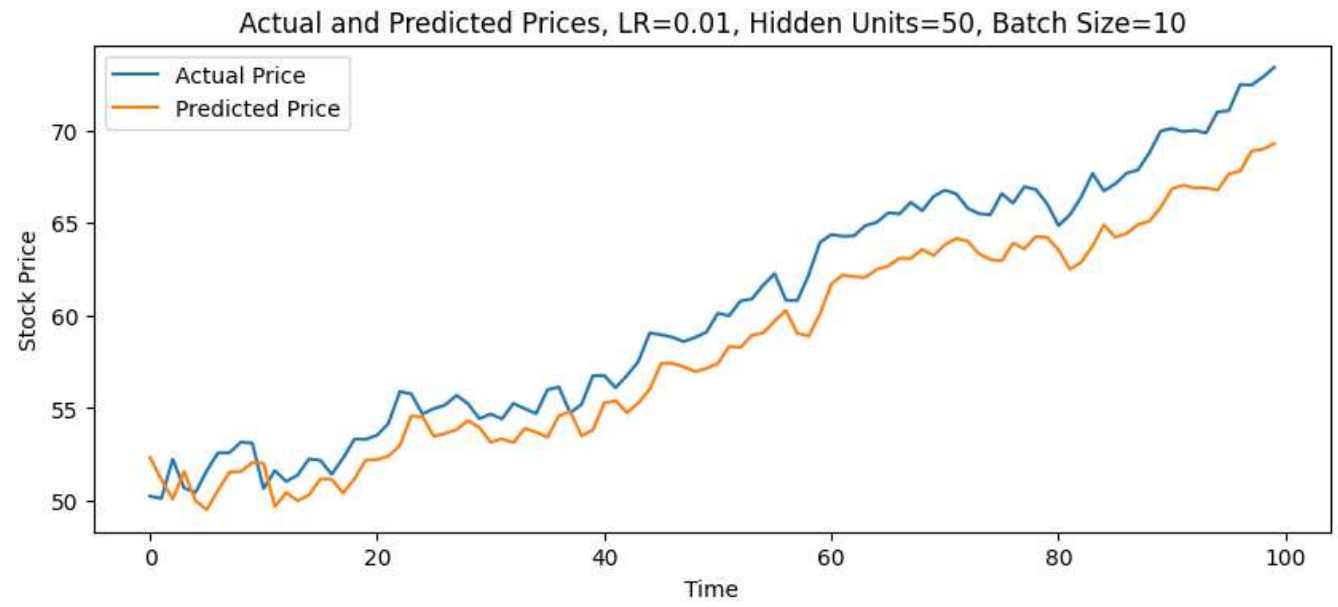
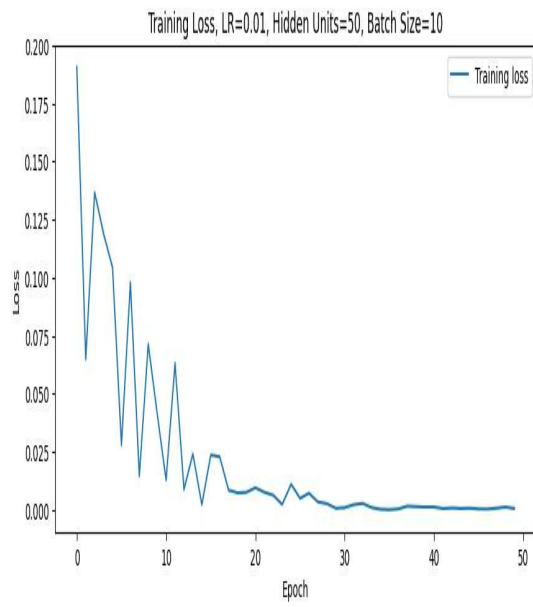


Actual and Predicted Prices, LR=0.001, Hidden Units=100, Batch Size=32





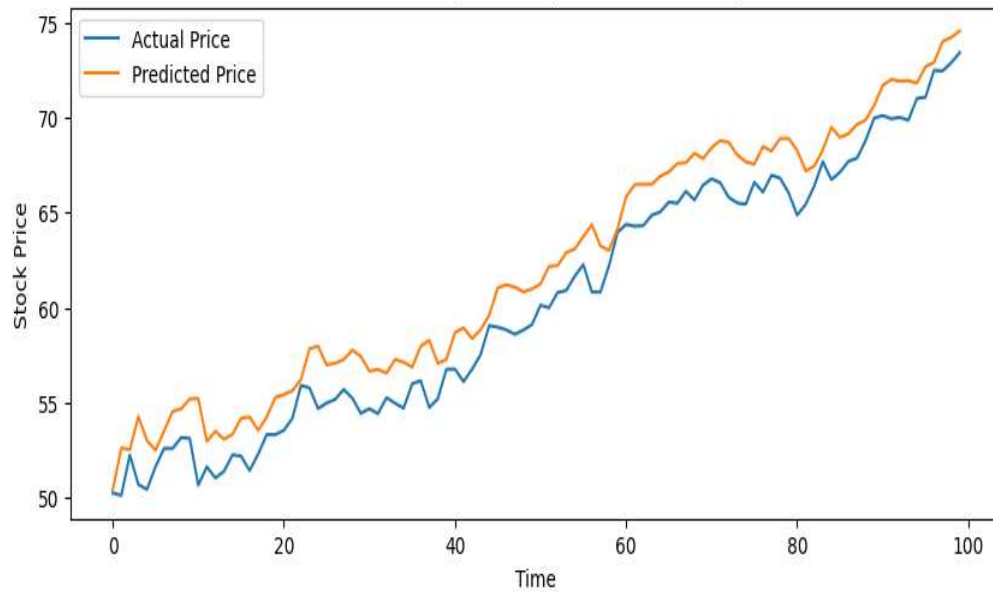
Results



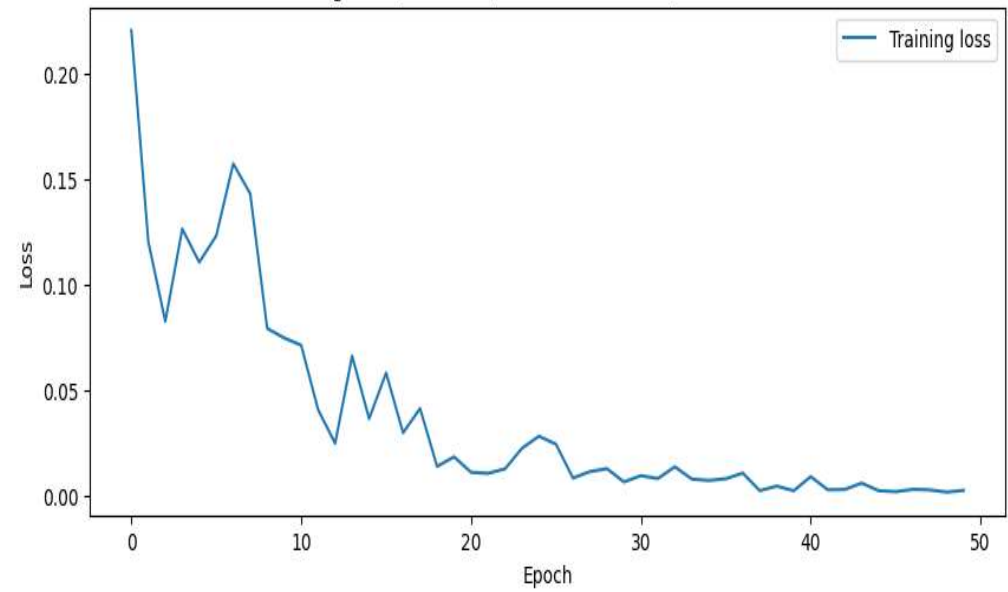
Results

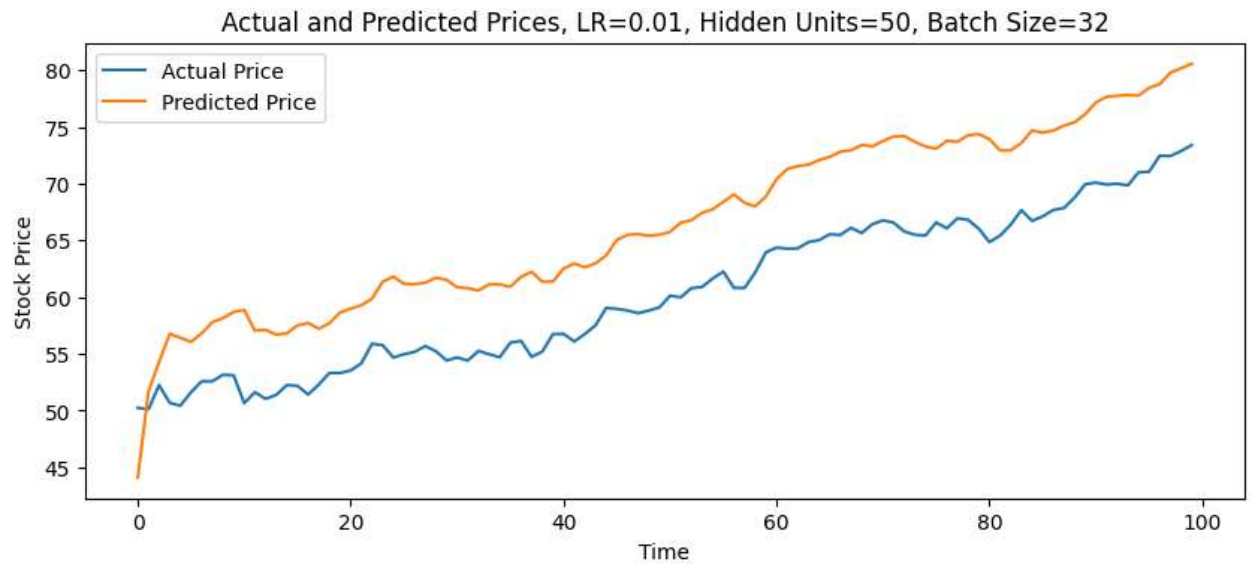
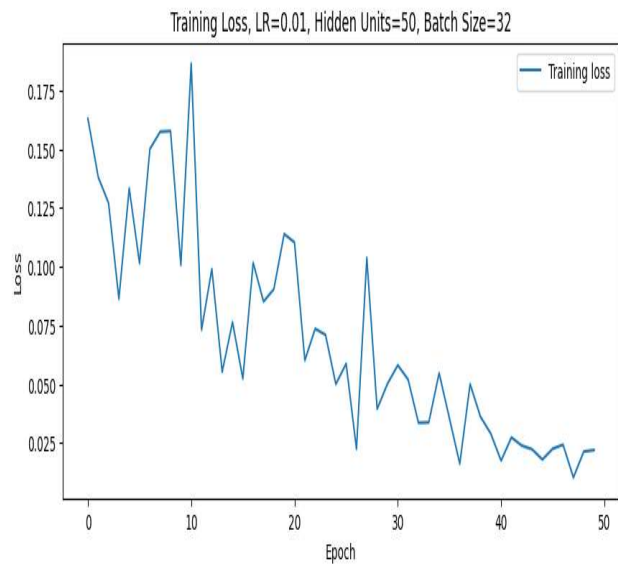
Results

Actual and Predicted Prices, LR=0.01, Hidden Units=50, Batch Size=16



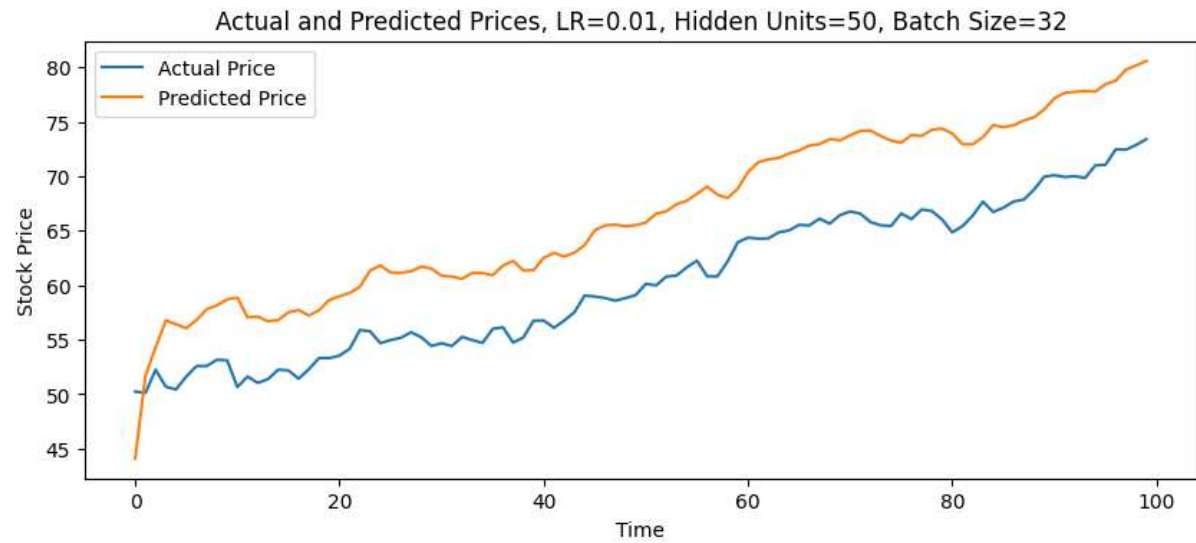
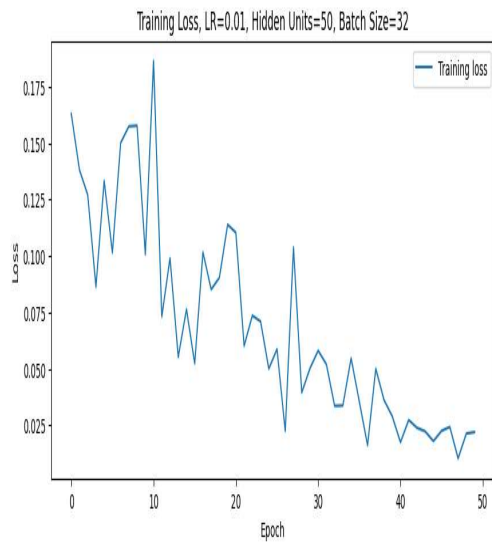
Training Loss, LR=0.01, Hidden Units=50, Batch Size=16



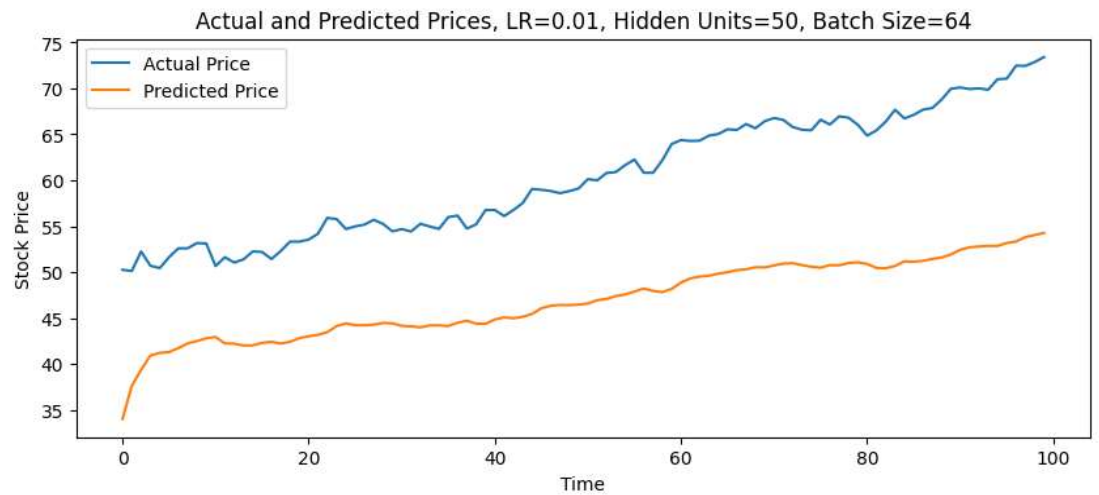
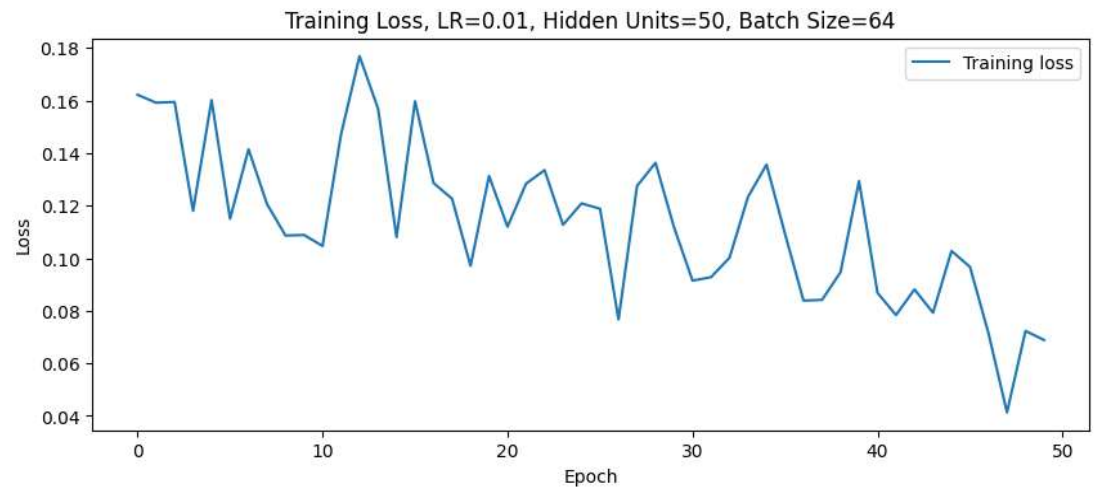


Results

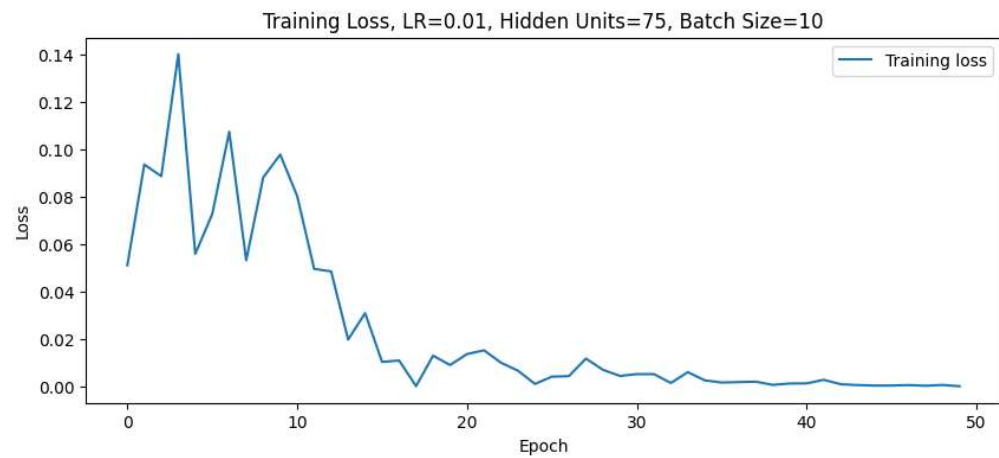
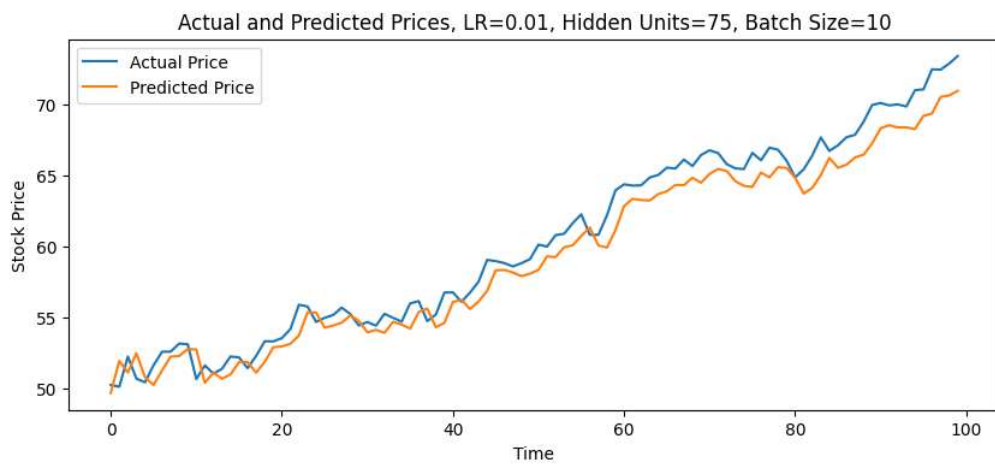
Results



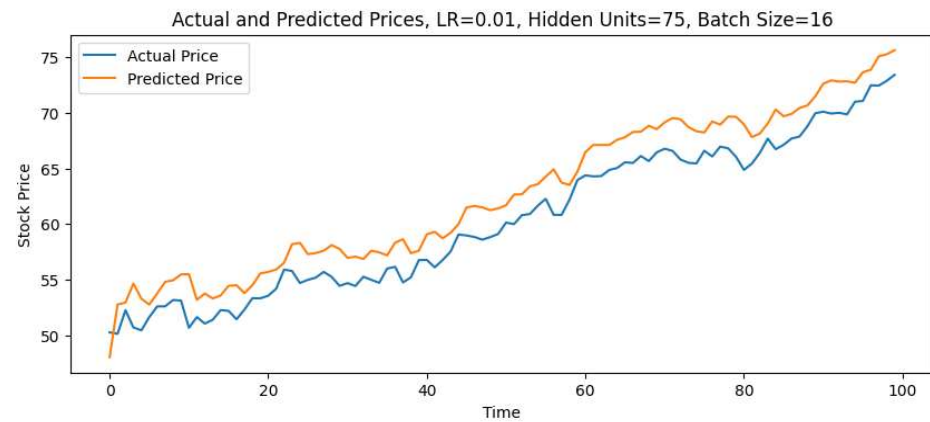
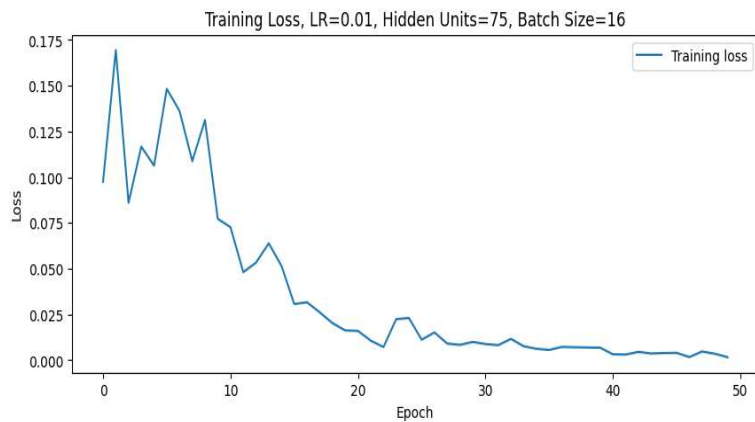
Results



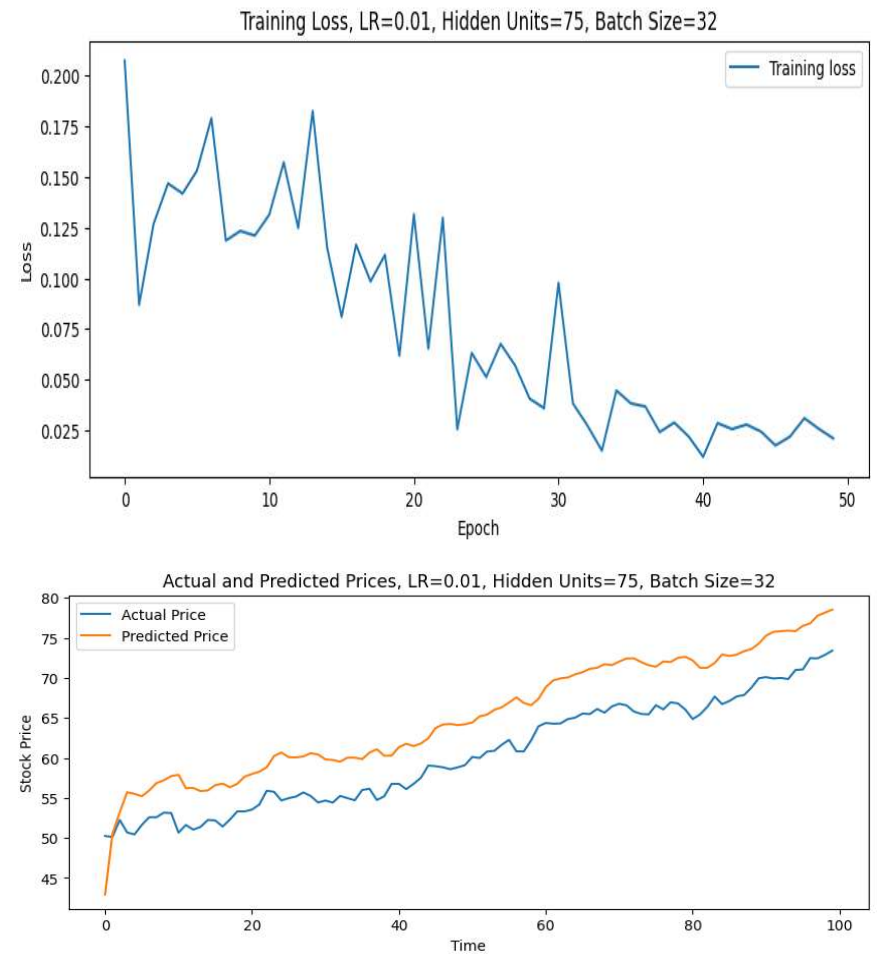
Results



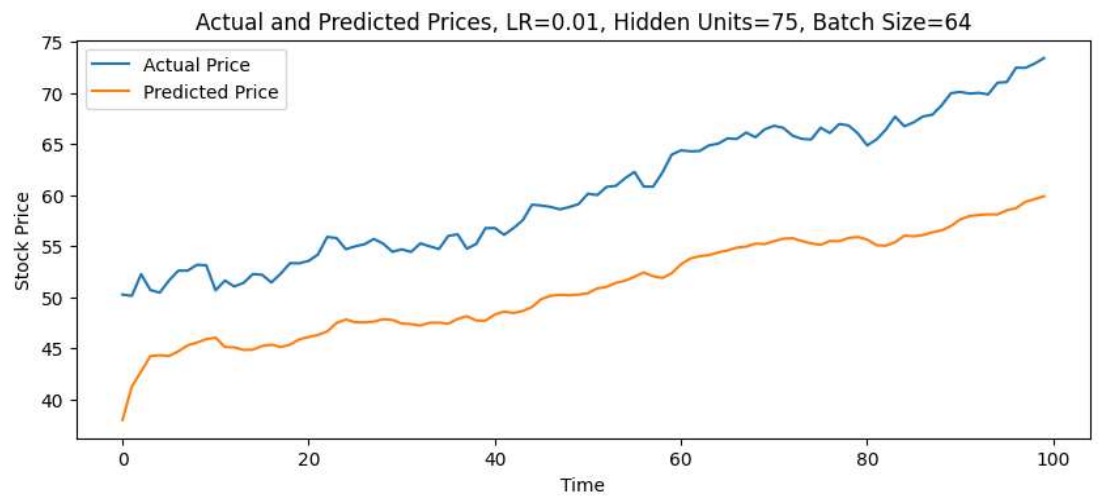
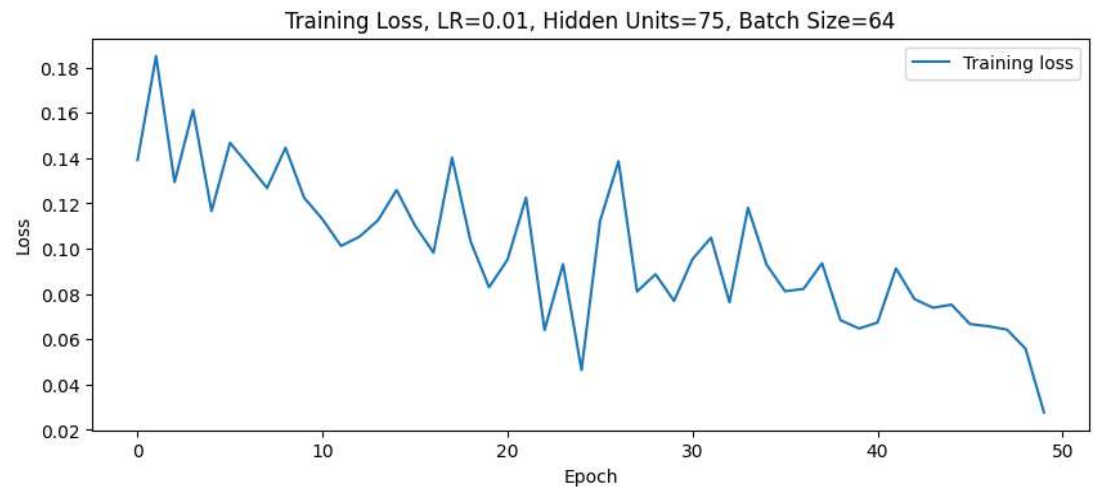
Results



Results

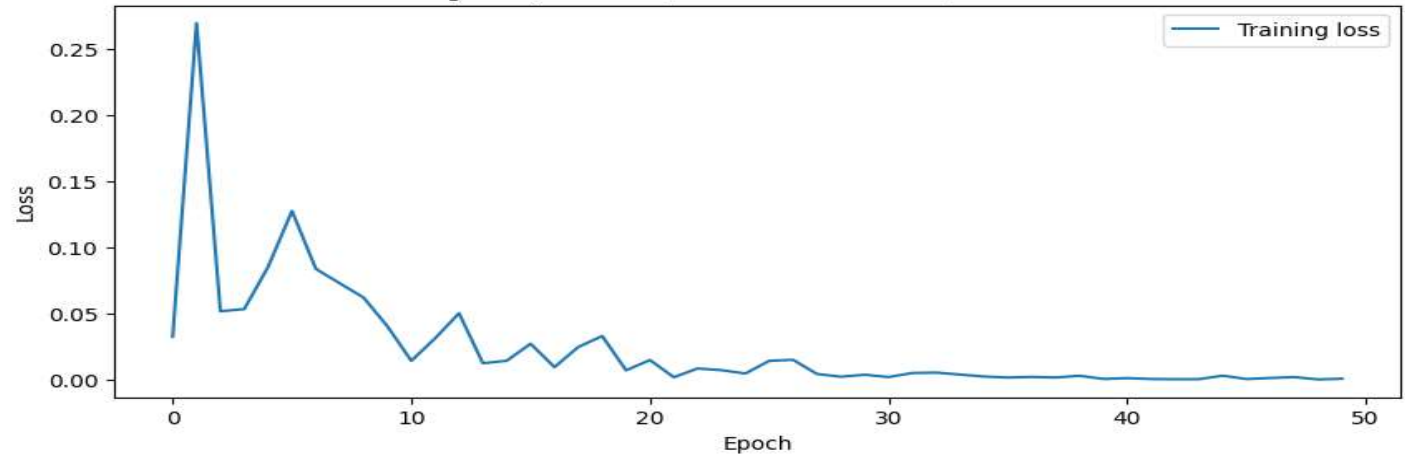


Results

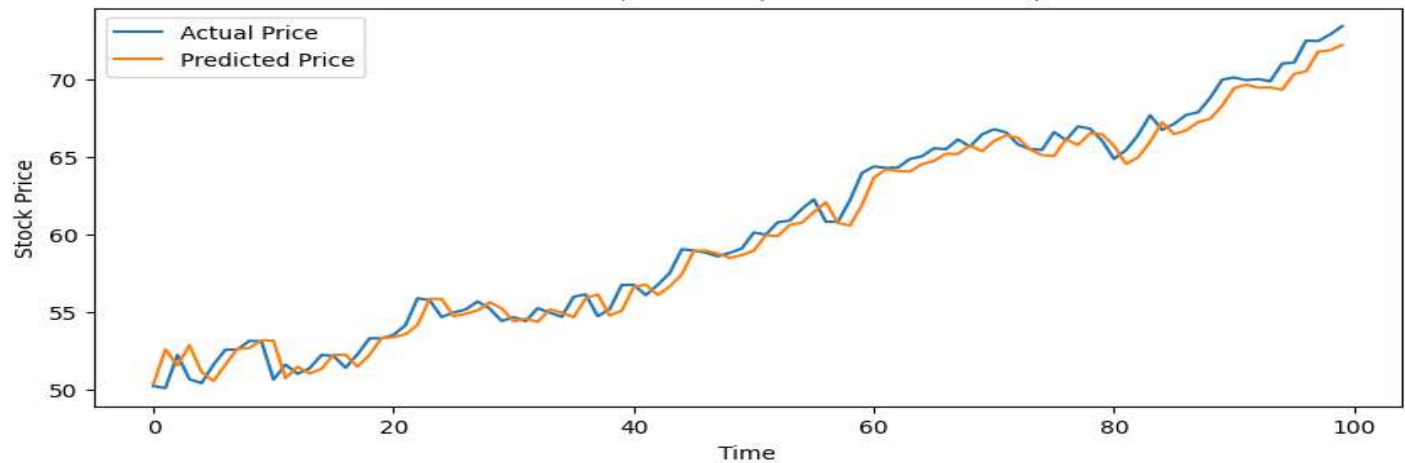


Results

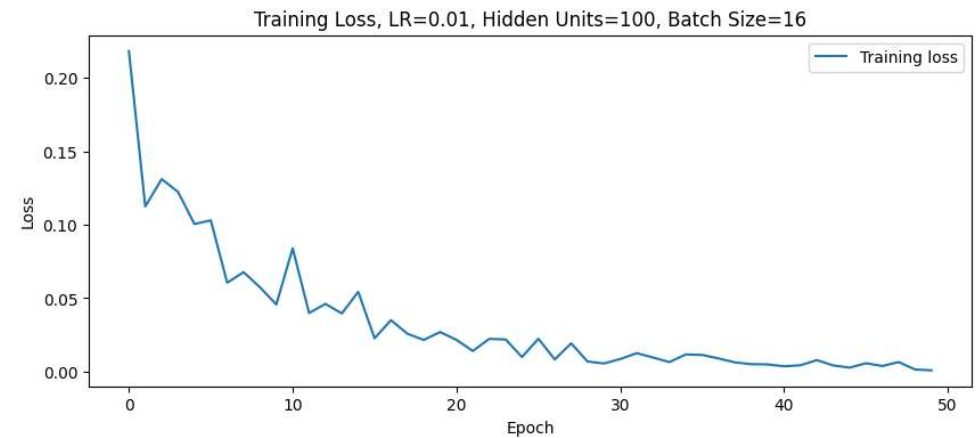
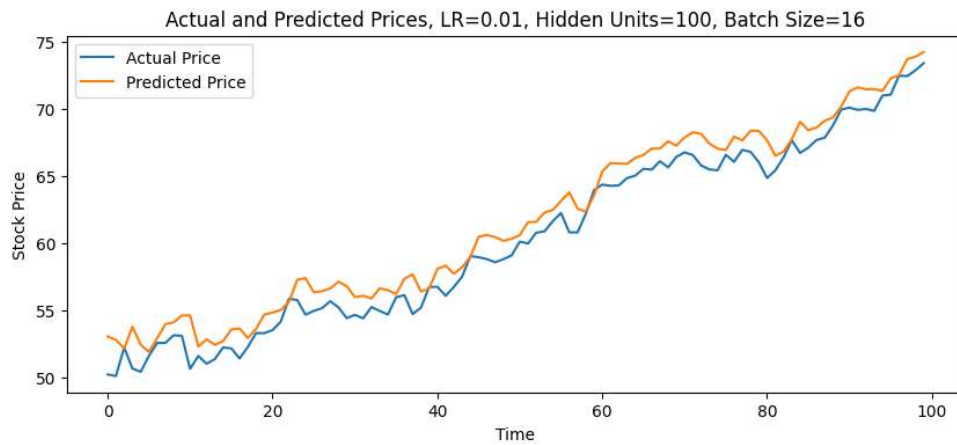
Training Loss, LR=0.01, Hidden Units=100, Batch Size=10



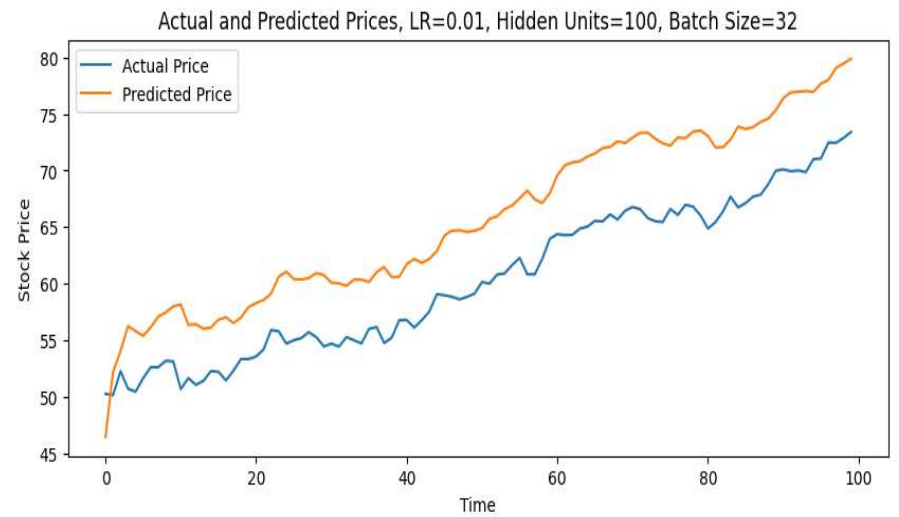
Actual and Predicted Prices, LR=0.01, Hidden Units=100, Batch Size=10



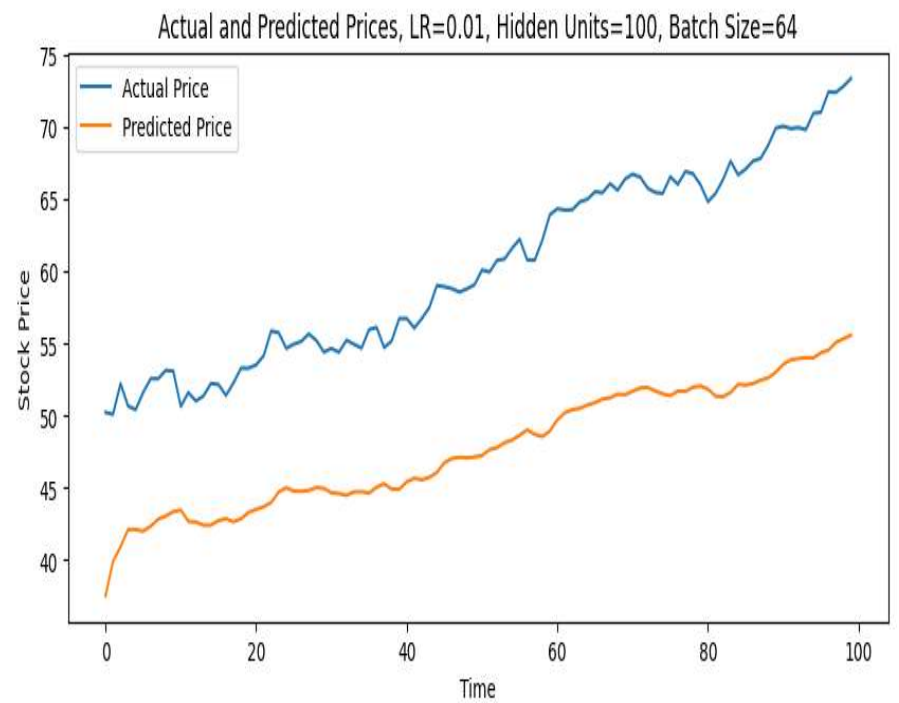
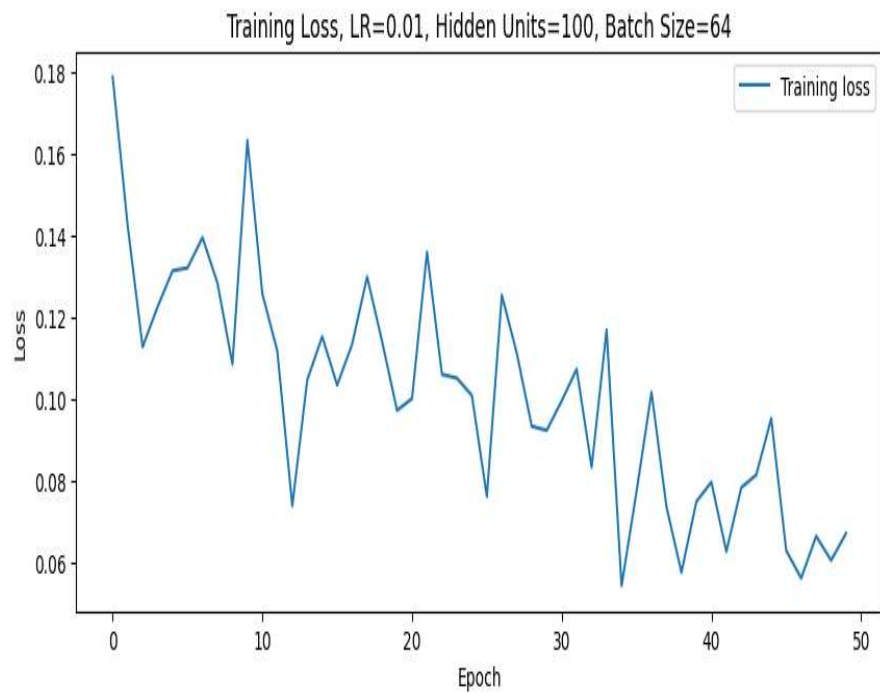
Results



Results

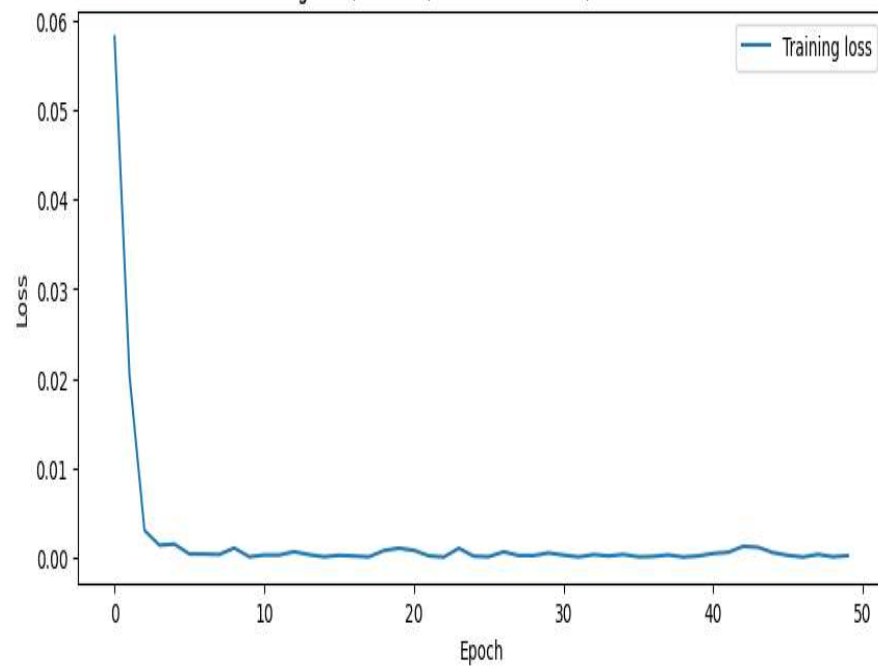


Results

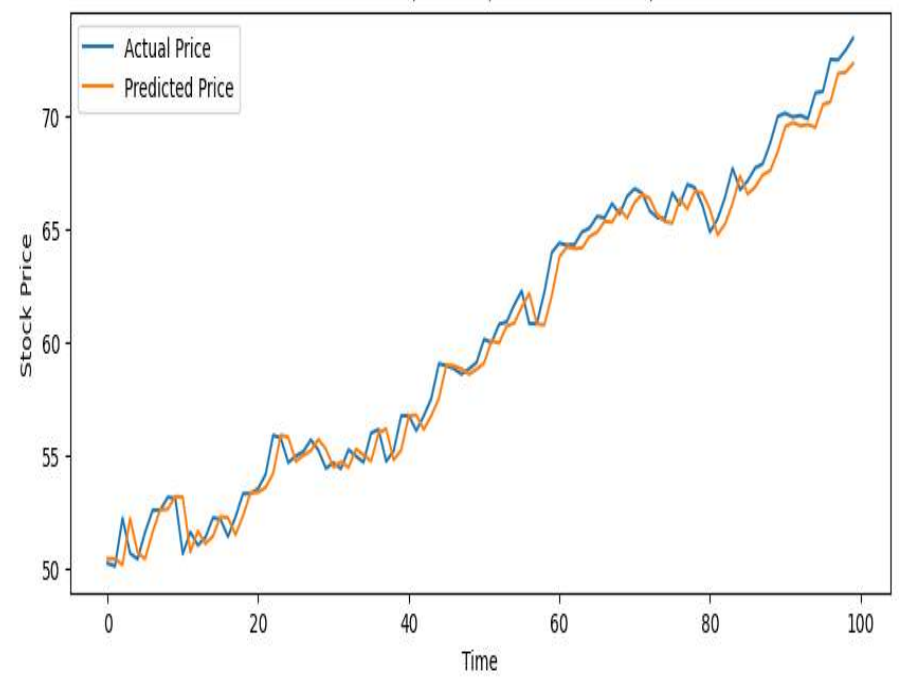


Results

Training Loss, LR=0.1, Hidden Units=50, Batch Size=10

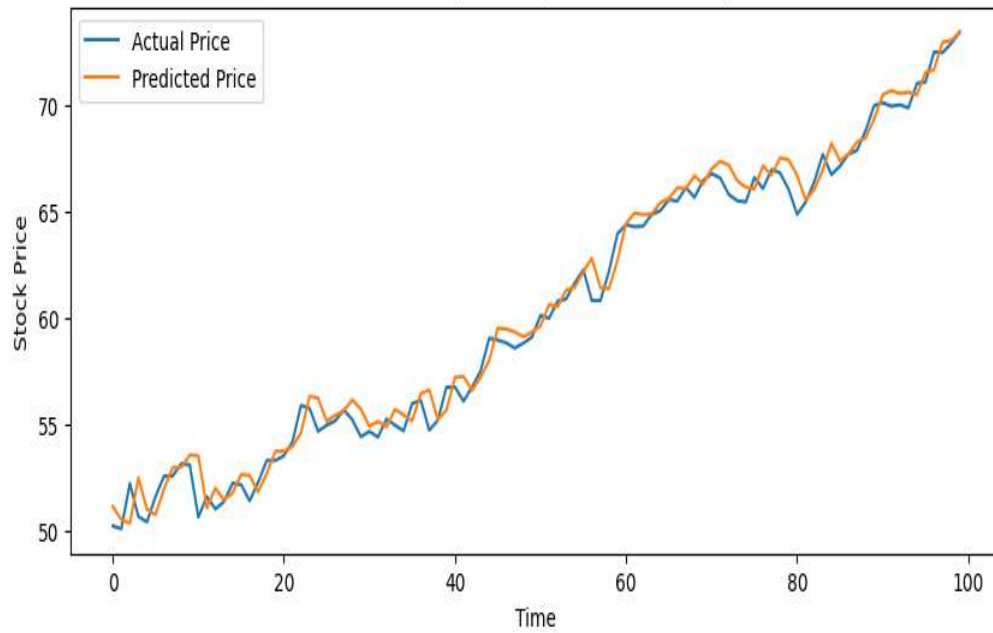


Actual and Predicted Prices, LR=0.1, Hidden Units=50, Batch Size=10

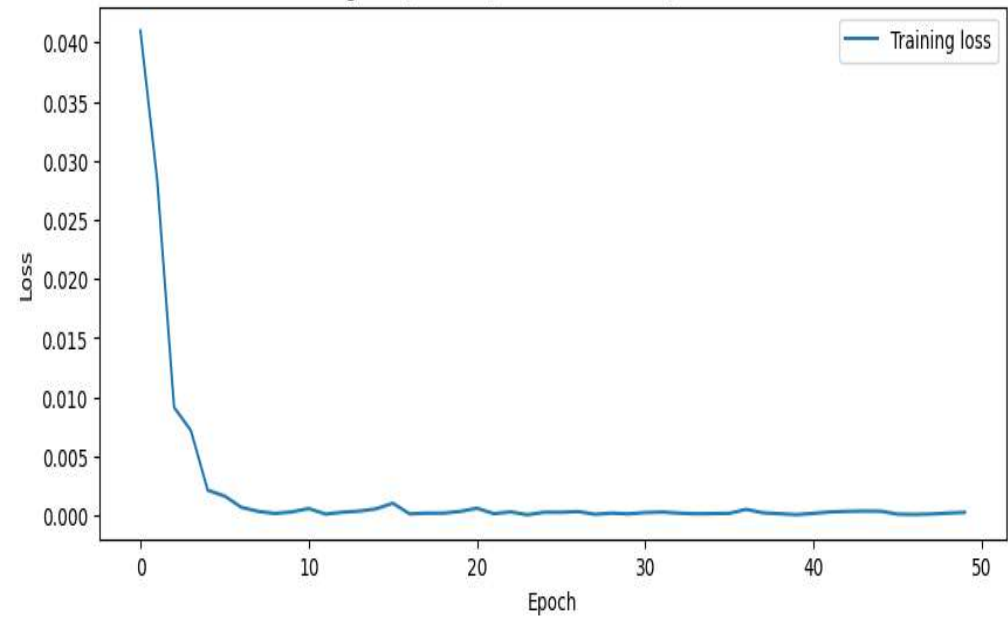


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=50, Batch Size=16

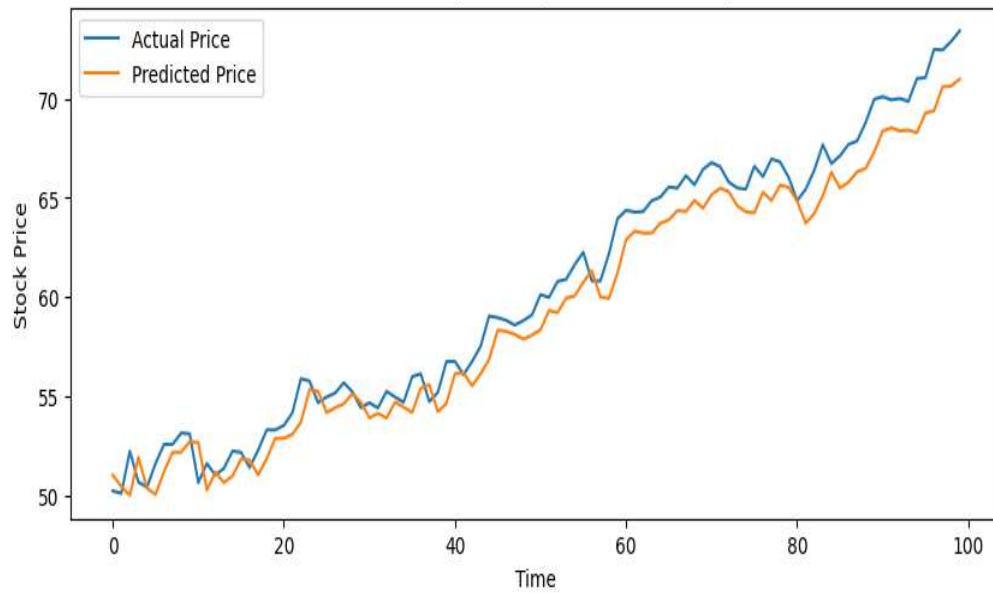


Training Loss, LR=0.1, Hidden Units=50, Batch Size=16

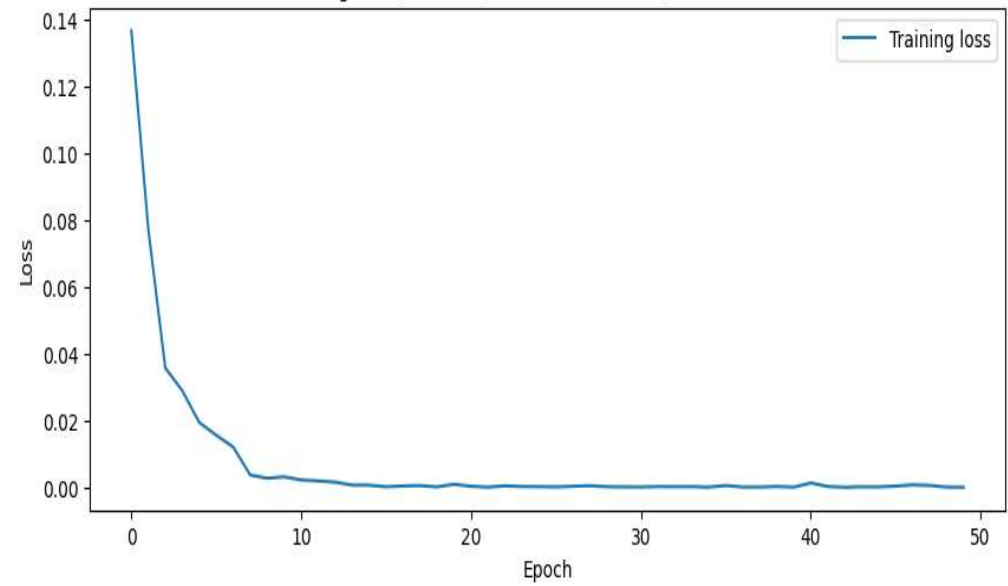


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=50, Batch Size=32

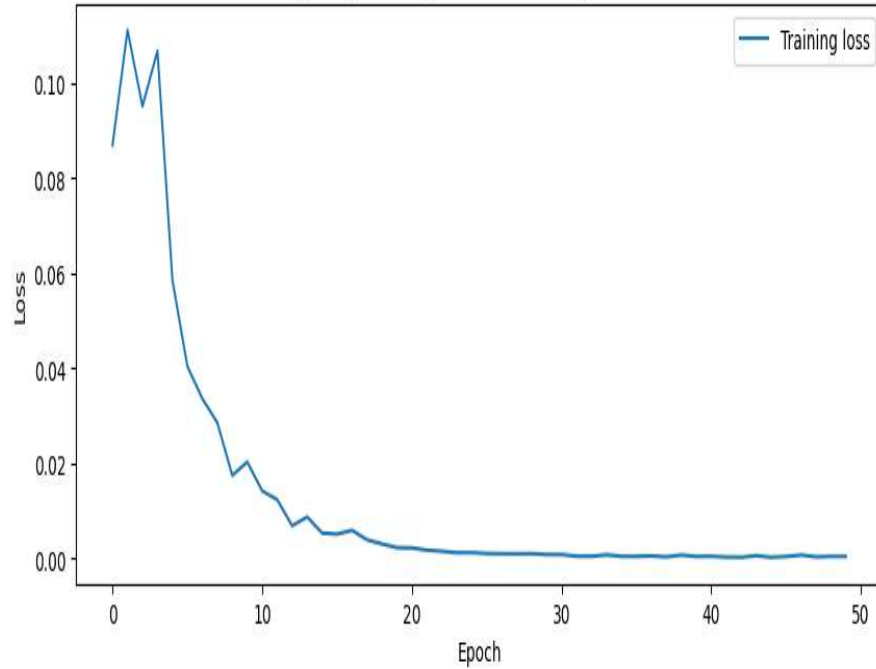


Training Loss, LR=0.1, Hidden Units=50, Batch Size=32

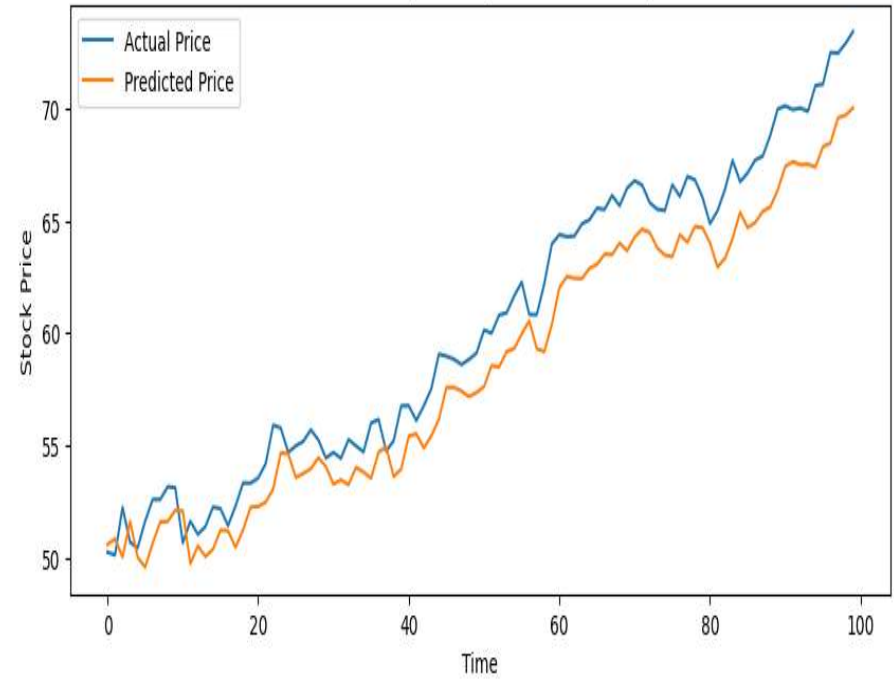


Results

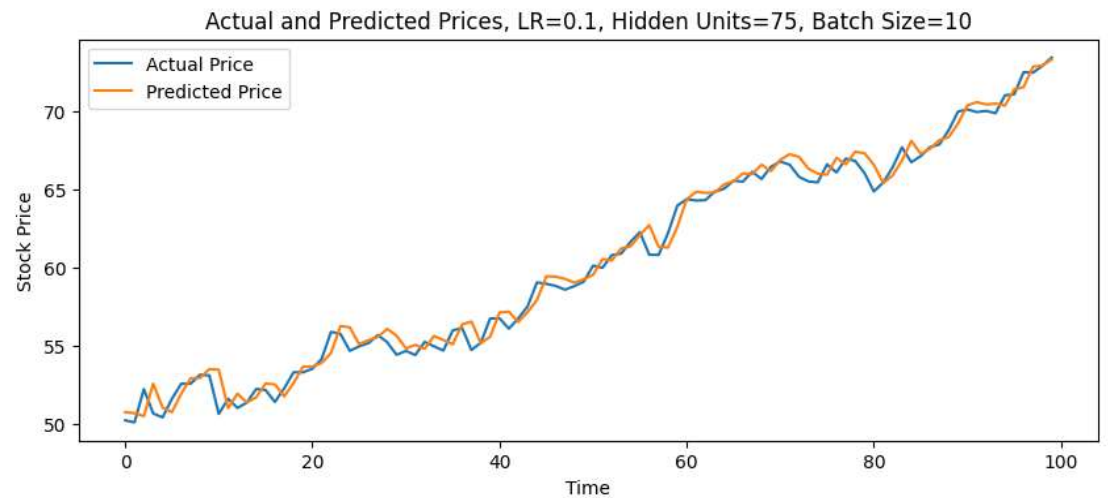
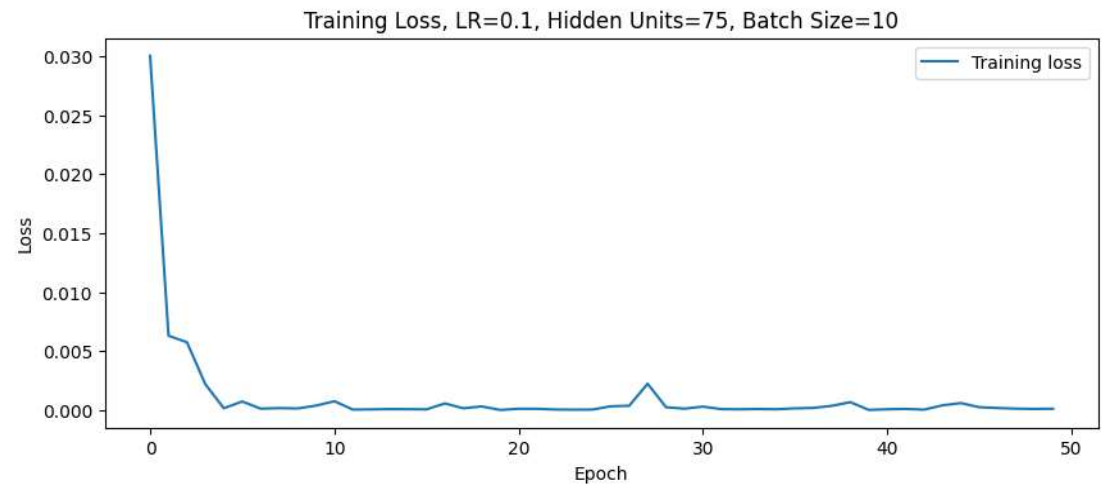
Training Loss, LR=0.1, Hidden Units=50, Batch Size=64



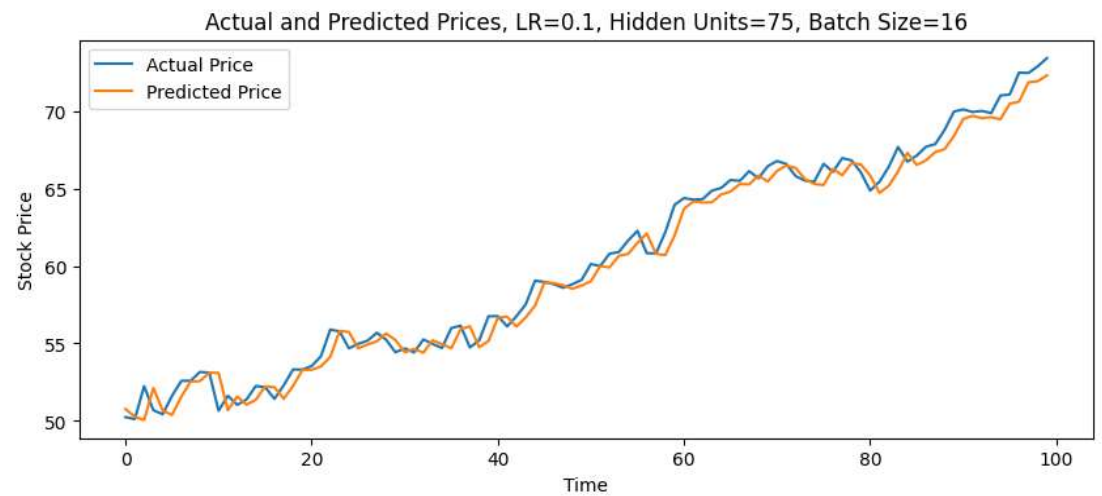
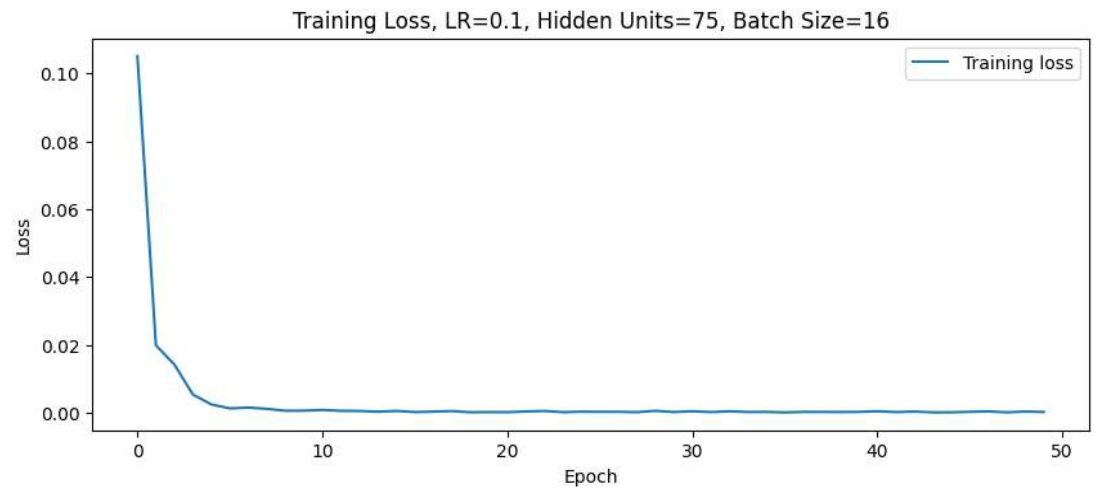
Actual and Predicted Prices, LR=0.1, Hidden Units=50, Batch Size=64



Results

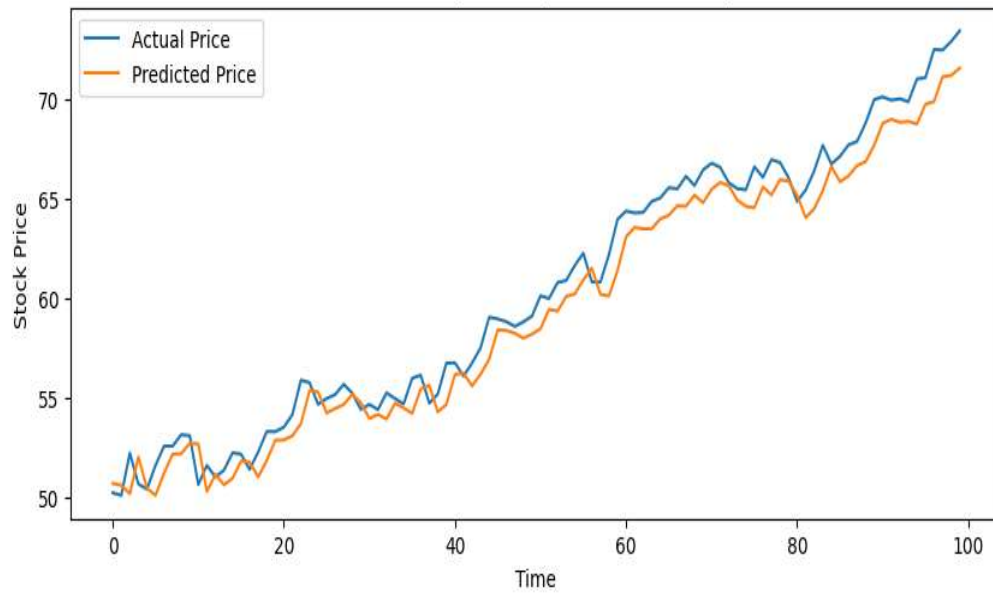


Results

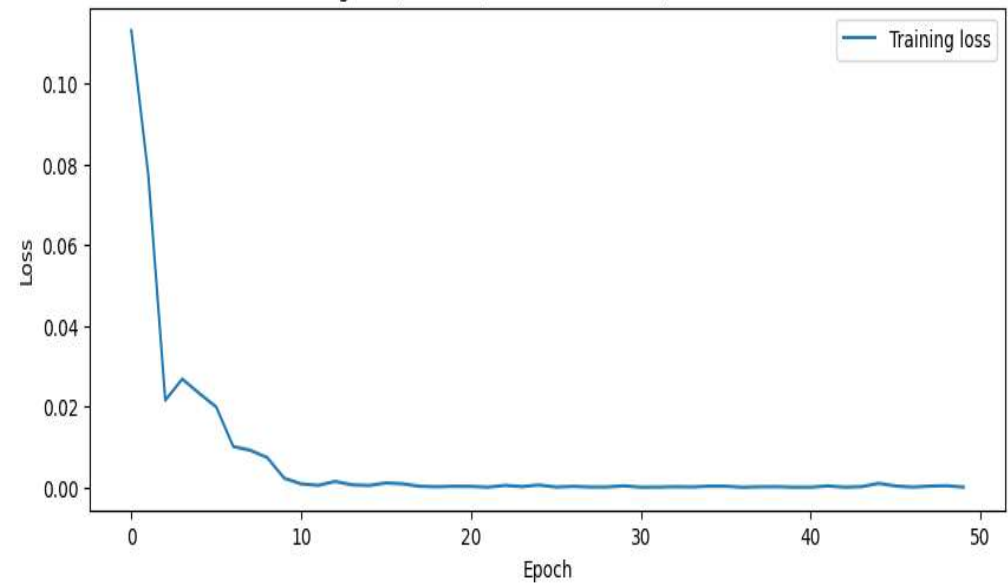


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=75, Batch Size=32

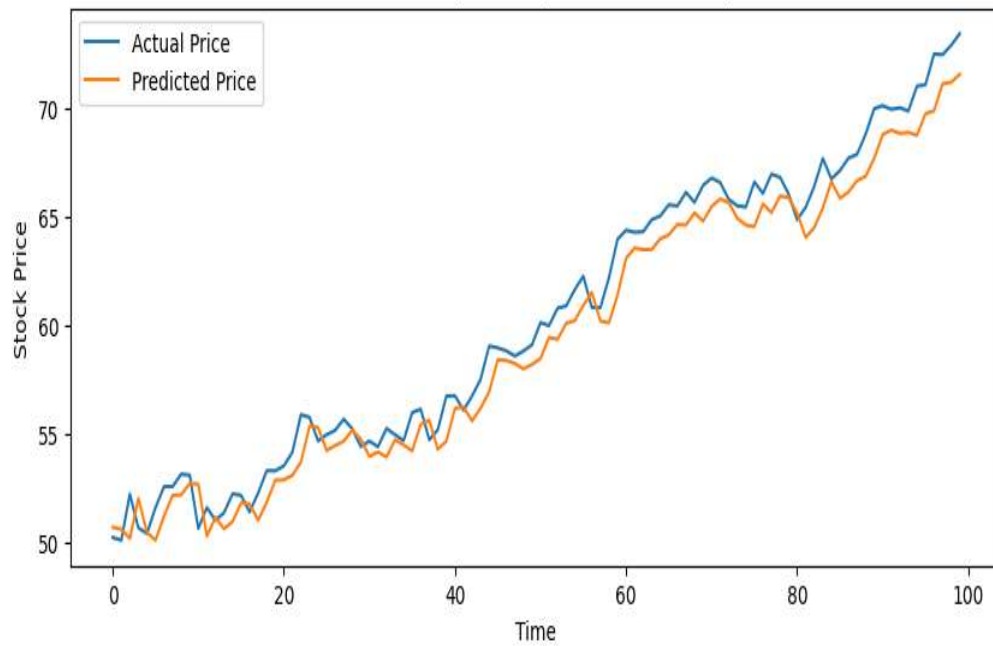


Training Loss, LR=0.1, Hidden Units=75, Batch Size=32

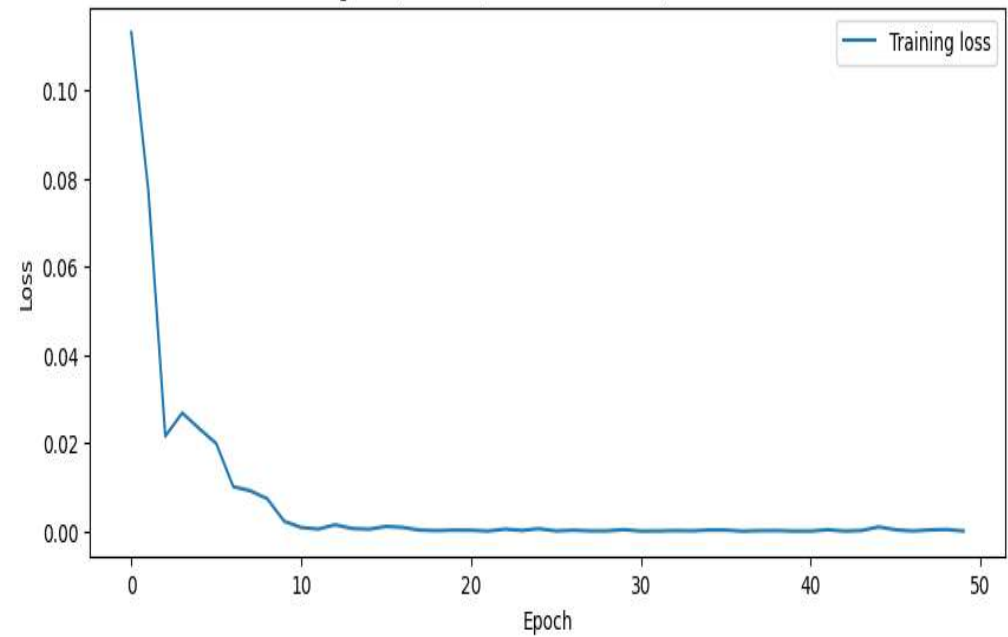


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=75, Batch Size=32

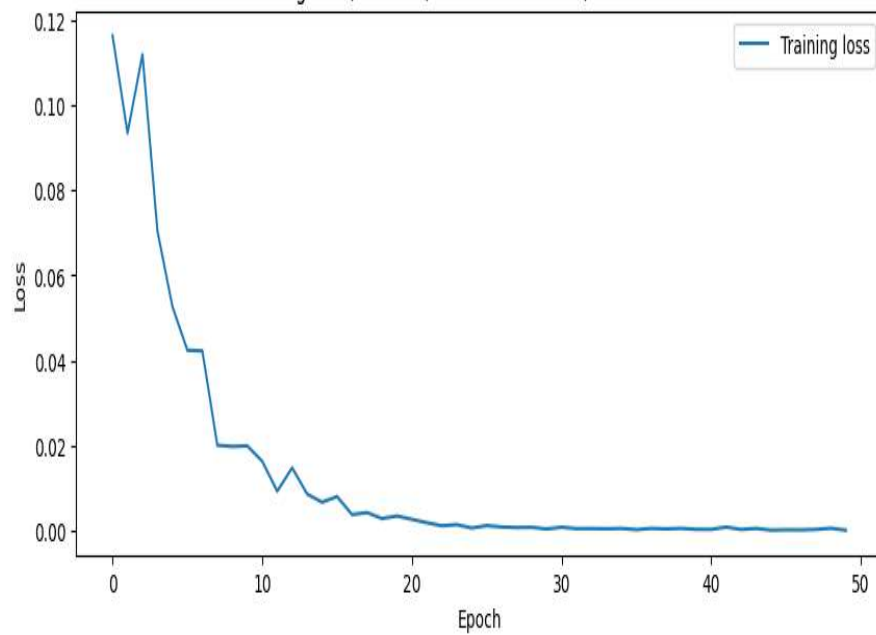


Training Loss, LR=0.1, Hidden Units=75, Batch Size=32

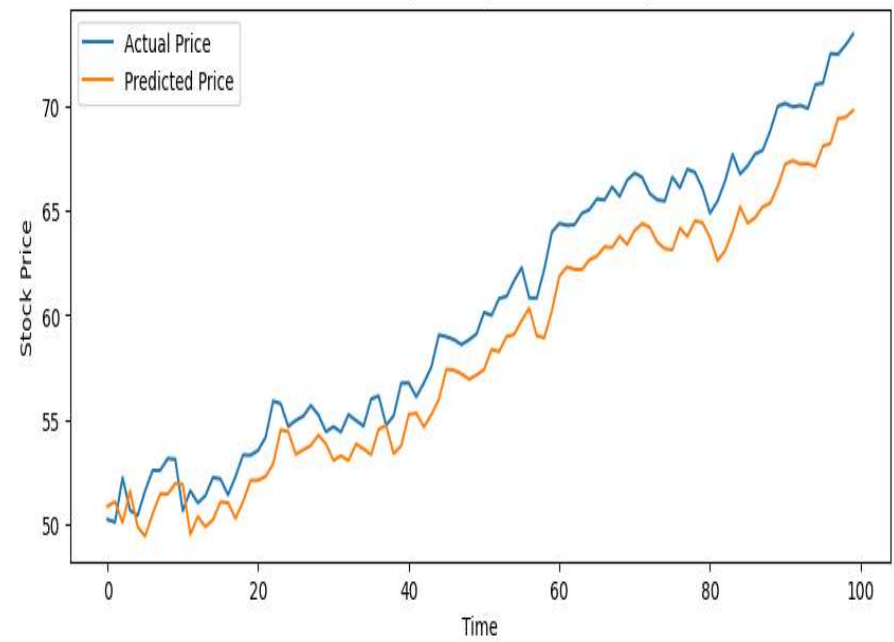


Results

Training Loss, LR=0.1, Hidden Units=75, Batch Size=64

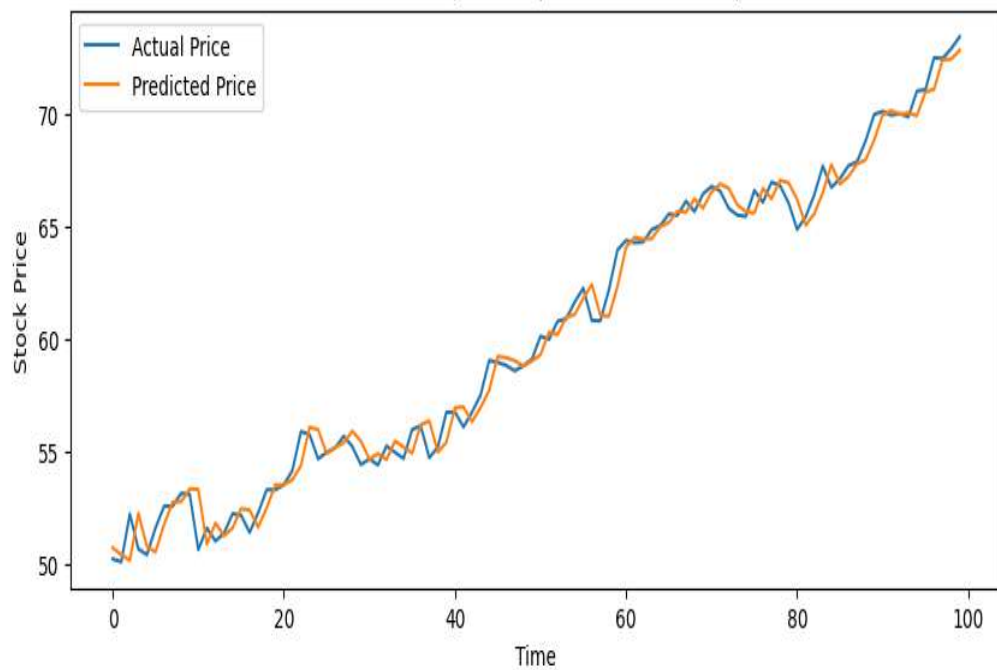


Actual and Predicted Prices, LR=0.1, Hidden Units=75, Batch Size=64

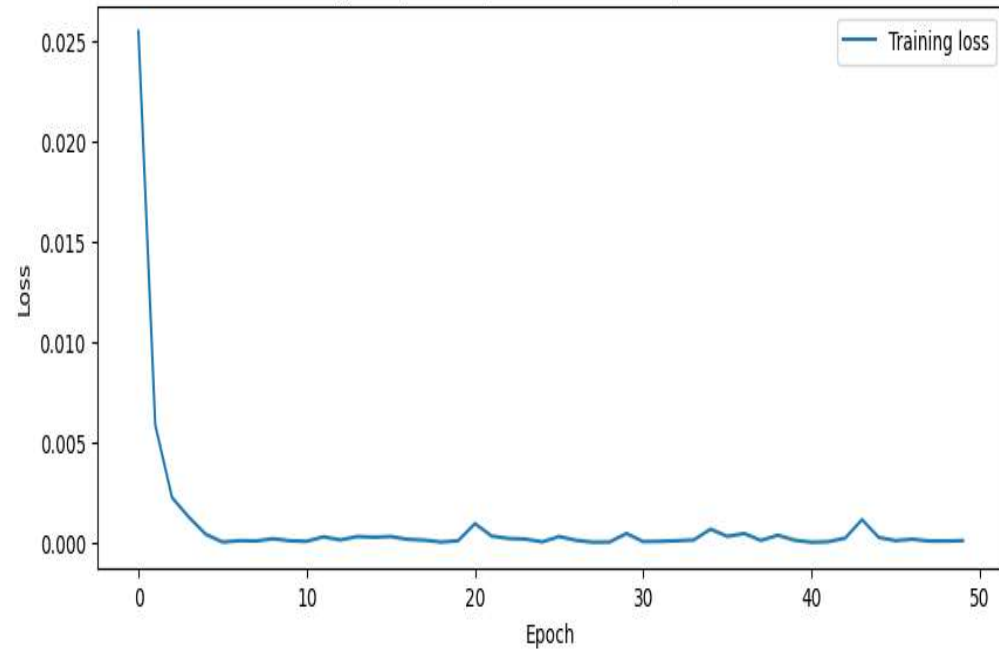


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=100, Batch Size=10

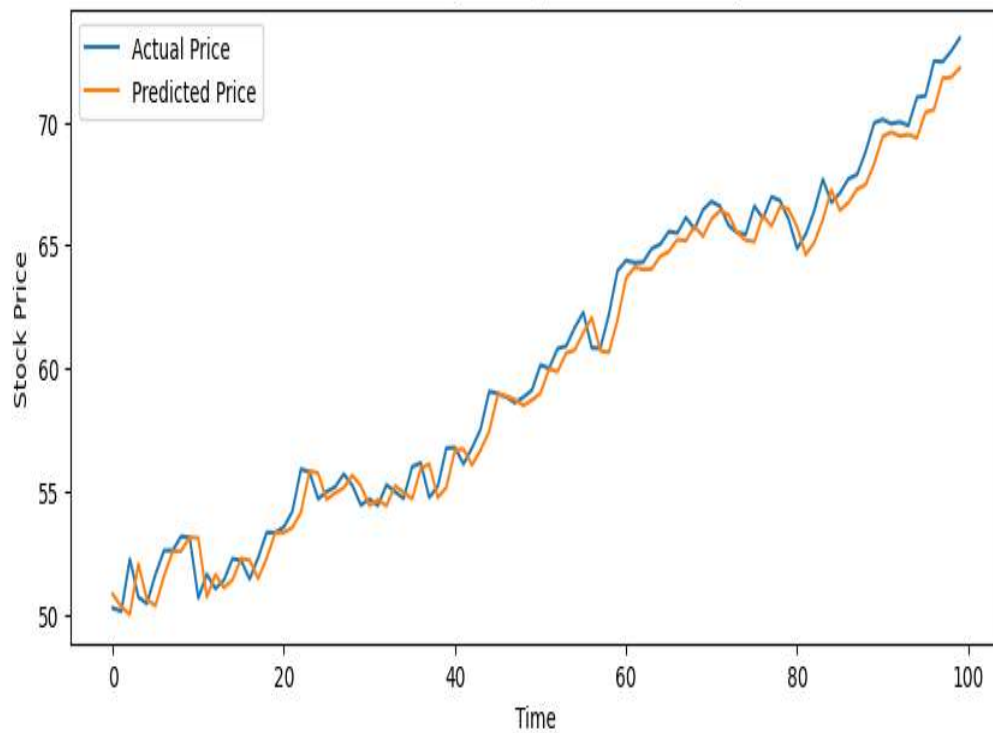


Training Loss, LR=0.1, Hidden Units=100, Batch Size=10

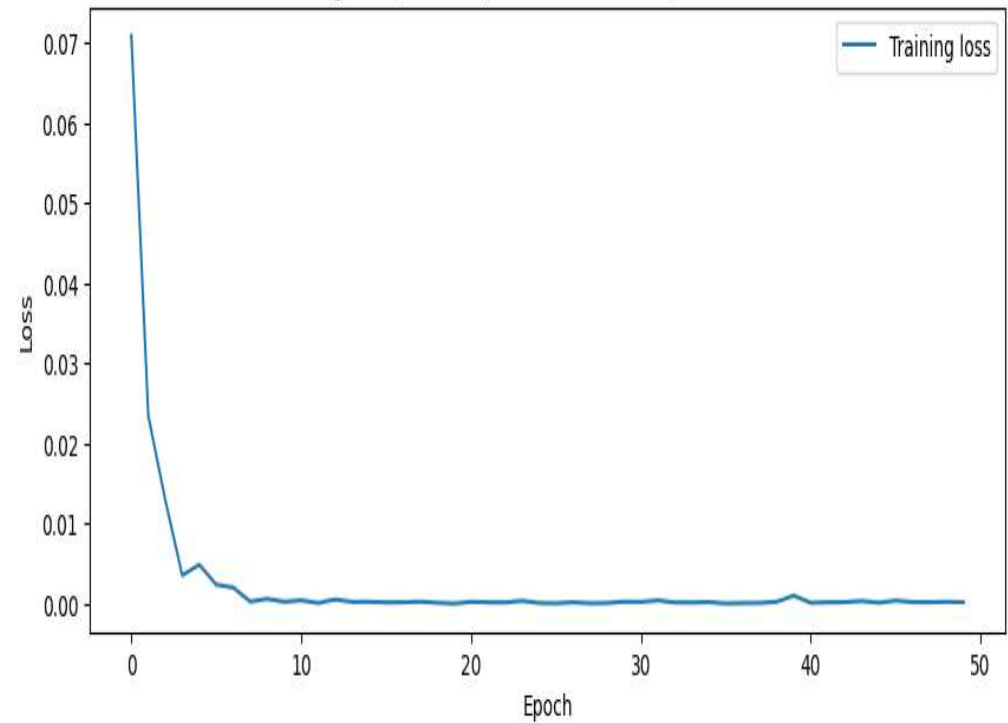


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=100, Batch Size=16

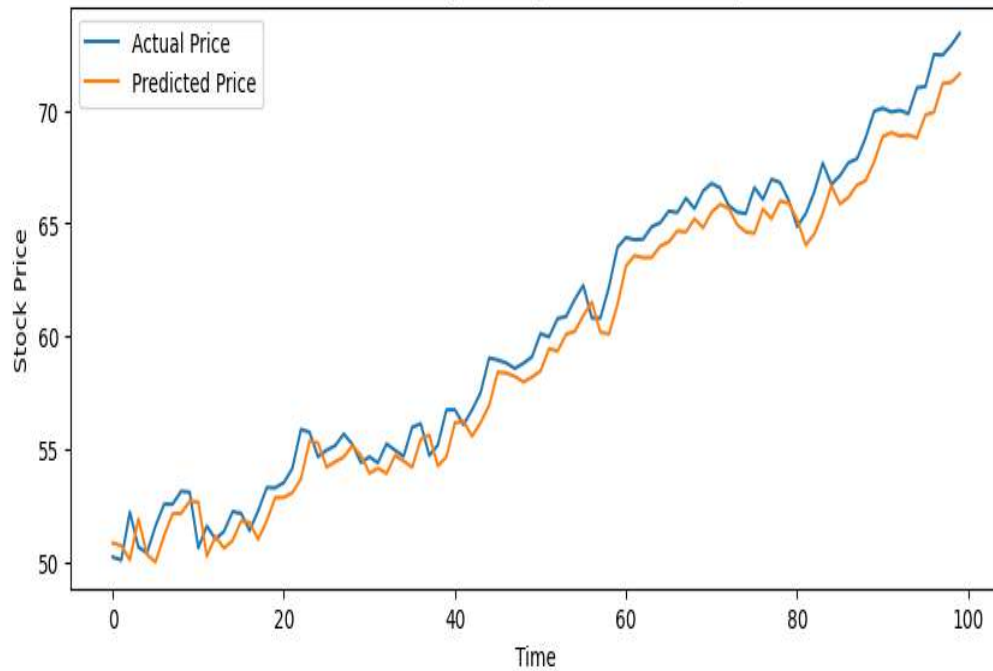


Training Loss, LR=0.1, Hidden Units=100, Batch Size=16

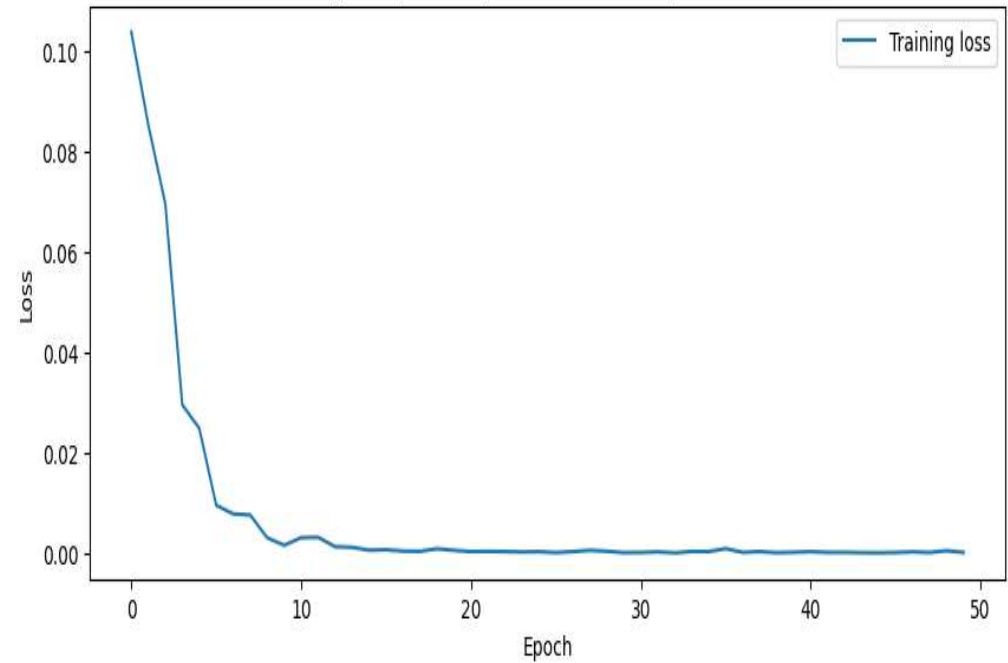


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=100, Batch Size=32

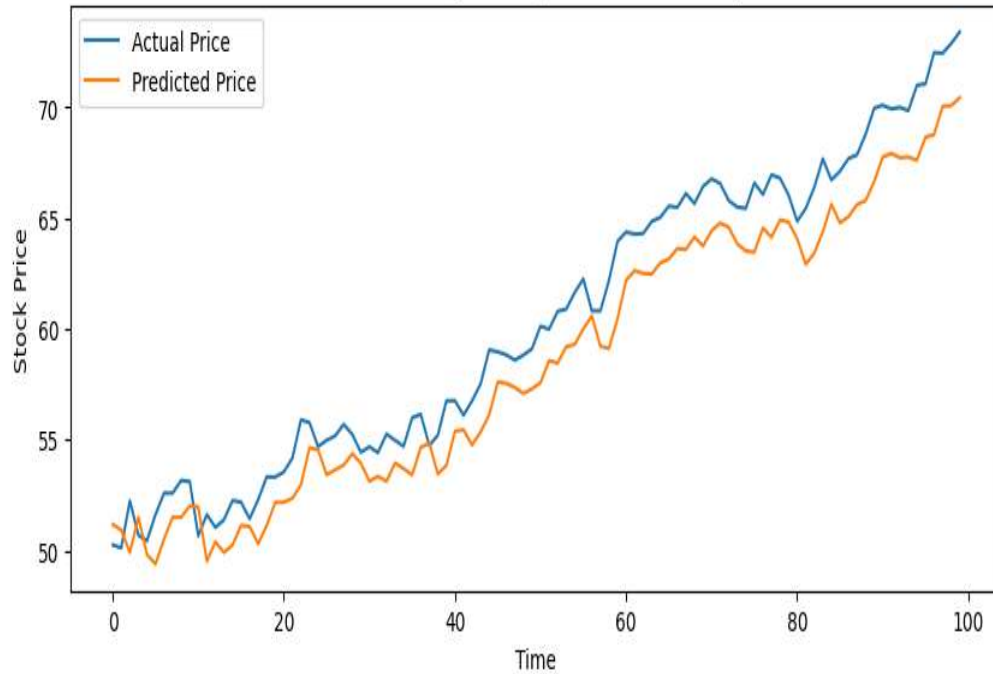


Training Loss, LR=0.1, Hidden Units=100, Batch Size=32

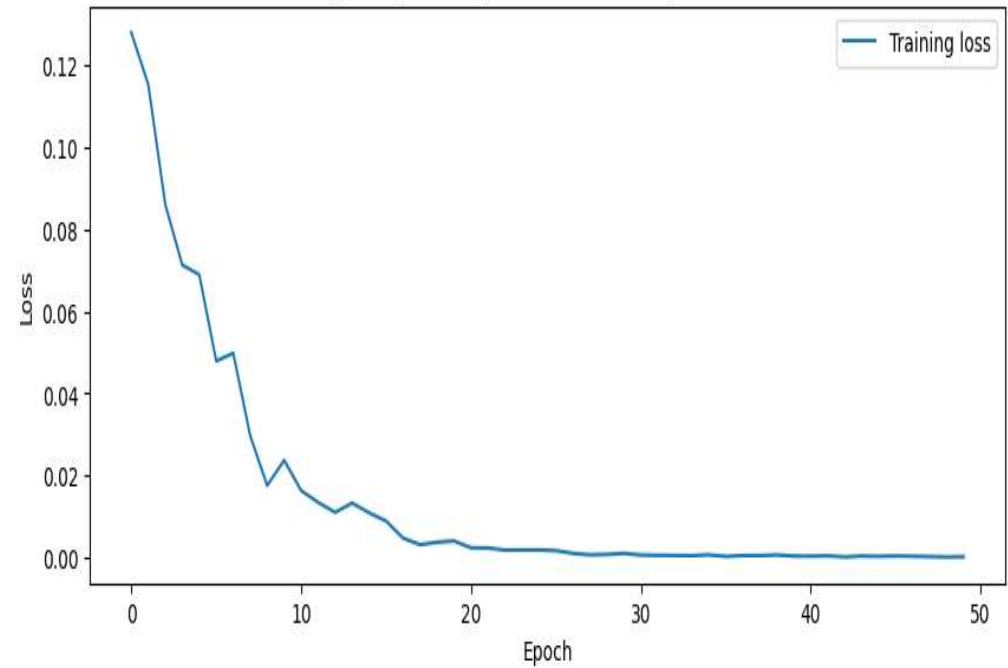


Results

Actual and Predicted Prices, LR=0.1, Hidden Units=100, Batch Size=64



Training Loss, LR=0.1, Hidden Units=100, Batch Size=64



Discussion

- The results demonstrate the performance of a machine learning model, specifically an LSTM network, across various training configurations. Each configuration is characterized by a unique combination of learning rate, hidden unit count, and batch size. The analysis of these results offers insights into how the model's learning dynamics are influenced by these hyperparameters.
- Loss Trajectory: Across all configurations, the loss generally decreases over the course of the 50 training epochs, indicating that the model is learning and improving its ability to fit the training data. However, the rate and stability of this loss reduction vary significantly depending on the hyperparameter settings.
- Learning Rate Impact: Lower learning rates (0.001) lead to a more gradual descent in loss, as the model makes smaller updates to the weights, requiring more epochs to converge. Conversely, **higher learning rates (0.01 and 0.1) result in faster loss decreases**, but also introduce more variability and occasional increases in loss, suggesting the model may be overshooting the optimal weights due to the larger weight updates.
- Batch Size Influence: Smaller batch sizes (10, 16) exhibit more fluctuations in the loss curve, likely due to the higher variance in gradient estimates from smaller data samples. **Larger batch sizes (32, 64), on the other hand, produce smoother and more stable loss curves**, as they provide a more accurate estimate of the gradient, but may potentially get stuck in local minima.
- Hidden Unit Effects: Increasing the number of hidden units generally allows the model to capture more complex patterns, as it has higher capacity. However, this does not necessarily lead to lower loss in all cases, as the increased complexity can also introduce the risk of overfitting, especially without appropriate regularization.
- Optimal Configurations: Based on the preliminary results, configurations with a moderate **learning rate (0.01) and larger batch sizes (32 or 64) appear to show more stable and lower final losses**, suggesting better generalization performance.
- Stability and Overfitting: Some configurations exhibit spikes in loss or erratic loss changes, which could indicate instability in the training dynamics or the model's tendency to overfit to noisy patterns in the data.
- Determining Optimal Epochs: **The ideal number of training epochs varies across configurations**, as the loss sometimes stabilizes or minimizes after a certain point. Continuing to train beyond this optimal point without improvements in validation metrics can lead to wasted computational resources and potential overfitting.
- To further optimize the model, we should consider employing cross-validation and more systematic hyperparameter tuning approaches, such as grid search or random search, to thoroughly explore the parameter space and identify the best-performing configuration based on validation performance. Monitoring both the training and validation losses is crucial to diagnose and prevent overfitting.